

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САРАТОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ИМЕНИ ГАГАРИНА Ю.А.»

Институт прикладных информационных технологий и коммуникаций

Кафедра «Прикладные информационные технологии»

Направление 09.03.02 «Информационные системы и технологии»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
«Разработка информационной системы витрины качества металла»

Студент (ка) _____ Парамонова Анна Сергеевна _____
фамилия, имя, отчество

курс 4 группа б1-ИФСТ-42

Руководитель

Доцент кафедры «Прикладные
информационные технологии»,
кандидат технических наук

01.06.2026

подпись, дата

А.В. Ермаков

Инициалы Фамилия

Допущен к защите

Приказ от «25» мая 2026 года № 1470-С

Зав. кафедрой «Прикладные
информационные технологии»,
кандидат технических наук, доцент

01.06.2026

подпись, дата

О.А. Торопова

Инициалы Фамилия

Саратов 2026 г

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САРАТОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ИМЕНИ ГАГАРИНА Ю.А.»

Институт прикладных информационных технологий и коммуникаций

Кафедра «Прикладные информационные технологии»

Направление 09.03.02 «Информационные системы и технологии»

ЗАДАНИЕ
на выпускную квалификационную работу

Студентке _____
Парамоновой Анне Сергеевне
фамилия, имя, отчество

Тема ВКР: «Разработка информационной системы витрины качества металла»

утверждена на заседании кафедры, протокол № 5 от «25» ноября 2025 г.

Дата защиты «_____» июня 2026 г.

Оценка защиты _____

Секретарь ГЭК Каликинская Е.Ю. _____
подпись

Саратов 2025 г

Целевая установка и исходные данные

Результатом выполнения выпускной квалификационной работы должна стать информационная система витрины качества металла, обеспечивающая консолидацию данных о производственных партиях из систем MES, SCADA и LIMS в рамках единого веб-приложения. Разрабатываемая система должна предоставлять пользователям инструменты мониторинга технологических процессов, контроля качества продукции и производственных показателей и оперативного выявления отклонений. Система должна обеспечивать ролевое разграничение доступа к информации и формирование отчетности.

Исходными данными являются требования к системам управления качеством согласно стандартам ISO 9001:2015 и ISO/IEC 17025:2017; данные о производственных процессах металлургического предприятия; архитектурные решения для веб-приложений.

№	перечень чертежей, подлежащих разработке	формат, кол-во
	<i>Чертежи не предусмотрены</i>	

Руководитель

Доцент кафедры «Прикладные
информационные технологии»,
кандидат технических наук

25.11.2025

А.В. Ермаков

должность, ученая степень, уч. звание

подпись, дата

Инициалы Фамилия

Содержание расчетно-пояснительной записки

Во введении должна быть описана актуальность разработки системы, цели и задачи работы.

В первой главе необходимо провести анализ предметной области: изучить особенности металлургического производства, классы производственных систем, провести обзор и сравнительный анализ существующих аналогов.

Во второй главе необходимо описать проектирование информационной системы: сформулировать функциональные возможности, провести описание входных и выходных данных, процессов и потоков разрабатываемой системы.

Третья глава должна быть посвящена описанию практической реализации системы: выбору технологий, реализации базы данных, взаимодействия компонентов системы, серверной и клиентской частей информационной системы в рамках тестовых данных.

В заключении формируются выводы по результатам выполненной работы, достижению целей и задач разработки.

Основная рекомендуемая литература

1. ПАО «ГМК «Норильский никель». Годовой отчёт за 2023 год [Электронный ресурс]. — URL: <https://ar2023.nornickel.ru>;
2. ПАО «ГМК «Норильский никель». Схема производства металлов [Электронный ресурс]. — URL: <https://ar2022.nornickel.ru/ru/business-overview/production-flow.html>;
3. Интегрированные системы проектирования и управления. SCADA : учебное пособие / Х. Н. Музипов, О. Н. Кузяков, С. А. Хохрин [и др.]. — Санкт-Петербург : Лань, 2022. — 408 с. — ISBN 978-5-8114-3265-3. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/213209>;
4. Федотов, А. В. Компьютерное управление в производственных системах : Учебное пособие для вузов / А. В. Федотов, В. Г. Хомченко. — 2-е изд., стер. — Санкт-Петербург : Лань, 2021. — 620 с. — ISBN 978-5-8114-8065-4. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/171424>;
5. Boyar, K., Pham, A., Swantek, S., Ward, G., Herman, G. Laboratory Information Management Systems (LIMS) // Cannabis Laboratory Fundamentals / ed. by S. R. Opie. — Cham : Springer, 2021. — P. 183–210. — DOI: 10.1007/978-3-030-62716-4_7.

Руководитель

Доцент кафедры «Прикладные
информационные технологии»,
кандидат технических наук

должность, ученая степень, уч. звание

25.11.2025

подпись, дата

А.В. Ермаков

Инициалы Фамилия

Задание принял к исполнению:

25.11.2025

подпись, дата

А.С. Парамонова

Инициалы Фамилия

УТВЕРЖДАЮ:
Руководитель ВКР
Ермаков А.В.

подпись

25.11.2025

КАЛЕНДАРНЫЙ ГРАФИК
работы над ВКР

№ п/п	Разделы, темы и их содержание	% от общего объема работы	Дата выполнения по графику	Отметка о выполнении, дата фактического выполнения, подпись руководителя
1	Анализ предметной области и аналогов	15	23.04.26	
2	Проектирование архитектуры системы	15	29.04.26	
3	Выбор технологий реализации	10	02.05.26	
4	Разработка базы данных	10	04.05.26	
5	Реализация серверной части системы	25	10.05.26	
6	Реализация клиентской части системы	20	20.05.26	
7	Оформление пояснительной записки, подготовка презентации	5	01.06.26	

Студент

25.11.2025

А.С. Парамонова

подпись, дата

Инициалы Фамилия

РЕФЕРАТ

Пояснительная записка на 65 страниц, 22 иллюстрации, 11 таблиц, 30 источников, 1 приложение.

Ключевые слова: витрина данных, управление качеством, контроль качества, металлургическое производство, MES, SCADA, LIMS.

Выпускная квалификационная работа посвящена разработке информационной системы, предназначенной для централизованного представления данных о производственных партиях и результатах контроля качества металлопродукции.

В рамках практической части были разработаны механизмы индексации и партиционирования для эффективной работы запросов к базе данных, асинхронного общения сервера и сервиса-эмулятора данных с помощью брокера сообщений. Реализована серверная часть приложения, обеспечивающая управление доступом к ресурсам, обработку пользовательских запросов, интеграцию с внешними источниками и формирование отчётности. Разработана клиентская часть, предоставляющая специализированные интерфейсы для различных категорий пользователей предприятия.

В результате выполнения выпускной квалификационной работы разработана информационная система витрины качества металла, обеспечивающая консолидацию данных, ролевую дифференциацию представления производственной информации, автоматическое обнаружение отклонений с доставкой уведомлений в режиме реального времени и генерацию PDF-отчётов по производственным партиям. Все компоненты системы контейнеризированы для упрощения развертывания.

Для разработки программного обеспечения были использованы следующие технологии: React 19, Java 21, Spring Boot 3, Apache Kafka, WebSocket/STOMP, PostgreSQL 18, iText 5, Docker.

АННОТАЦИЯ

Выпускная квалификационная работа посвящена разработке информационной системы витрины качества металла.

Актуальность разработки обусловлена необходимостью централизованного контроля качества продукции и повышения оперативности принятия решений в условиях современного металлургического производства.

В работе выполнены анализ предметной области, проектирование архитектуры системы и разработка программной составляющей системы.

Разработанная информационная система обеспечивает мониторинг производственных партий, контроль качества продукции, автоматическое выявление отклонений технологических параметров с оперативным информированием пользователей и формирование отчётности.

Работа состоит из пояснительной записки на 65 страниц, содержит 22 иллюстрации, 11 таблиц, 30 источников и 1 приложение.

ABSTRACT

The final qualifying work is devoted to the development of a metal quality dashboard information system.

The relevance of the development is due to the need for centralized product quality control and increased decision-making efficiency in modern metallurgical production.

The work includes the analysis of the subject area, the design of the system architecture and the development of the software component of the system.

The developed information system provides monitoring of production batches, product quality control, automatic detection of deviations in technological parameters with prompt user information and reporting.

The work consists of a 65-page explanatory note, contains 22 illustrations, 11 tables, 30 sources and 1 annex.

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САРАТОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
ИМЕНИ ГАГАРИНА Ю.А.»

ОТЗЫВ

руководителя о выпускной квалификационной работе

Разработка информационной системы витрины качества металла

наименование темы выпускной квалификационной работы

студентки 4 курса Института прикладных информационных технологий и
коммуникаций Парамоновой Анны Сергеевны

фамилия, имя, отчество

прошедшей обучение по направлению 09.03.02 «Информационные системы и
технологии» профиль «Информационные системы и технологии»

Выпускная квалификационная работа Парамоновой А.С. посвящена вопросам разработки информационной системы обеспечивающей консолидацию информации о производстве металла на предприятии из различных источников. Работа выполнена на базе запроса от ПАО «ГМК „Норильский никель“».

В ходе работы Анна Сергеевна продемонстрировала высокую степень самостоятельности и проявила способность анализировать и систематизировать информацию, ответственно подходила к выполнению задач. Особенно необходимо отметить её готовность разбираться в совершенно новой прикладной области при условиях некоторой пассивности заказчика. При этом можно отметить некоторые огрехи в процессе проектирования баз данных, что не снижает качества выполненной работы.

Содержание выпускной квалификационной работы изложено технически грамотно, пояснительная записка оформлена в соответствии с требованиями. Оригинальность составляет 96,12%

Работа не содержит информации, составляющей государственную, интеллектуальную, коммерческую тайну, или другую информацию, не подлежащую публикации без согласования с правообладателем.

Учитывая вышеизложенное, считаю, что Парамонова Анна Сергеевна достойна присвоения квалификации бакалавр по направлению 09.03.02 «Информационные системы и технологии», работа заслуживает оценки «отлично».

Руководитель

Доцент кафедры «Прикладные
информационные технологии»,
кандидат технических наук

01.06.2026 А.В. Ермаков

должность, ученая степень, уч. звание

подпись, дата

Инициалы Фамилия

СПРАВКА

о результатах проверки текстового документа
на наличие заимствований

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ образовательное учреждение
высшего образования "Саратовский
государственный технический университет
имени Гагарина Ю.А."

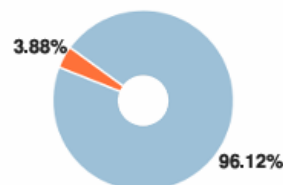
ПРОВЕРКА ВЫПОЛНЕНА В СИСТЕМЕ АНТИПЛАГИАТ.ВУЗ

Автор работы: Парамонова Анна Сергеевна
Самоцитирование
рассчитано для: Парамонова Анна Сергеевна
Название работы: 220812_Б1-ИФСТ-42_2026_1
Тип работы: Выпускная квалификационная работа
Подразделение:

РЕЗУЛЬТАТЫ

СОВПАДЕНИЯ	3.88%
ОРИГИНАЛЬНОСТЬ	96.12%
ЦИТИРОВАНИЯ	0%
САМОЦИТИРОВАНИЯ	0%
ИИ-КОНТЕНТ	0%

ДАТА ПОСЛЕДНЕЙ ПРОВЕРКИ: 11.06.2026



Структура документа: Проверенные разделы: основная часть с.29-61, введение с.1-11, методы с.12-28, выводы с.62-64

Модули поиска: Профессиональная лексика. АПК и биотех; PubMed; Перефразирования по базе публикаций открытого доступа PubMed; Переводные заимствования; Профессиональная лексика. Юриспруденция; Патенты СССР, РФ, СНГ; Сводная коллекция научных работ Беларуси; Медицина; Публикации РГБ; Перефразирования по Коллекции открытых публикаций международных издательств; Профессиональная лексика. Медицина; Шаблонные фразы; IEEE; СПС ГАРАНТ: аналитика; Кольцо вузов; Цитирование; СМИ России и СНГ; Коллекция НБУ; ИПС Адилет; Коллекция открытых публикаций международных издательств; Публикации eLIBRARY; Публикации РГБ (переводы и перефразирования); Публикации eLIBRARY (переводы и перефразирования); Перефразирования по СПС ГАРАНТ: аналитика; Перефразированные заимствования по коллекции Инт...

Работу проверил: Ермаков Александр Вадимович

ФИО проверяющего

Дата подписи:

Подпись проверяющего



Чтобы убедиться
в подлинности справки, используйте QR-код,
который содержит ссылку на отчет.

Ответ на вопрос, является ли обнаруженное заимствование
корректным, система оставляет на усмотрение проверяющего.
Предоставленная информация не подлежит использованию
в коммерческих целях.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	5
1.1 Описание предметной области	5
1.2 Обзор существующих решений.....	6
1.3 Сравнительный анализ существующих решений.....	10
1.4 Постановка задачи на разработку	12
2 ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ	14
2.1 Функциональные возможности системы.....	14
2.2 Описание данных системы.....	14
2.3 Описание процессов и потоков системы	17
2.4 Архитектура системы	24
3 ОПИСАНИЕ ПРАКТИЧЕСКОЙ РЕАЛИЗАЦИИ СИСТЕМЫ.....	26
3.1 Выбор технологий реализации	26
3.2 Реализация базы данных	29
3.3 Реализация взаимодействия компонентов системы	36
3.4 Реализация серверной части	39
3.5 Реализация клиентской части	49
3.6 Итоги реализации системы	63
ЗАКЛЮЧЕНИЕ	64
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	67
ПРИЛОЖЕНИЕ А.....	71

ВВЕДЕНИЕ

Информационная система разрабатывается для предприятия ПАО «ГМК „Норильский никель“» (далее – Норникель), одного из крупнейших мировых производителей цветных и драгоценных металлов. Для предприятий такого масштаба характерны высокие объёмы производственных данных, поступающих из различных автоматизированных систем управления технологическими процессами: систем управления производством (MES – Manufacturing Execution System), систем диспетчерского управления и сбора данных (SCADA – Supervisory Control and Data Acquisition), а также лабораторных информационных систем (LIMS – Laboratory Information Management System). Каждая из этих систем решает свои специфические задачи и хранит данные в собственных форматах и базах данных. Отсутствие интегрированного представления данных о качестве партий металла приводит к ситуации, при которой персонал предприятия вынужден ожидать получения информации от других сотрудников, не имея единой сводной «витрины» для быстрого принятия управленческих решений [1-3, 30].

В связи с этим актуальной становится задача создания единой информационной системы витрины качества металла, обеспечивающей:

- консолидацию данных из разнородных источников (MES, SCADA, LIMS) для получения полной картины состояния партий металла в реальном времени;
- дифференцированное представление информации в зависимости от роли пользователя – сотрудники лаборатории, производственный персонал и руководство имеют различные потребности в данных и различный уровень доступа к информации;
- механизм автоматического обнаружения отклонений показателей качества от нормативных значений с немедленным оповещением ответственных лиц.

Целью выпускной квалификационной работы является разработка

информационной системы витрины качества металла – веб-приложения, интегрирующего данные из систем MES, SCADA и LIMS и предоставляющего дифференцированные интерфейсы для различных категорий пользователей предприятия.

Для достижения поставленной цели необходимо решить следующие задачи:

- Провести анализ предметной области металлургического производства и задач управления качеством металлопродукции;
- Изучить существующие информационные системы и программные решения в области управления качеством продукции металлургических предприятий;
- Провести сравнительный анализ существующих решений и обосновать необходимость разработки собственной системы;
- Разработать функциональные и нефункциональные требования к проектируемой информационной системе;
- Разработать модель данных для хранения информации о партиях металла и результатах контроля качества;
- Спроектировать пользовательские интерфейсы для ролей: лаборатория, производство, руководство и администратор;
- Реализовать серверную часть системы;
- Реализовать клиентскую часть системы с ролевой моделью доступа;
- Разработать модуль мониторинга отклонений и систему уведомлений;
- Реализовать функцию автоматической генерации PDF-отчётов по партиям металла.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Описание предметной области

Металлургическое производство представляет собой сложный многостадийный технологический процесс, включающий переработку сырьевых материалов и получение металлической продукции заданных свойств. Управление качеством продукции охватывает всю цепочку производственных операций – от контроля входного сырья и полуфабрикатов до испытания готовой продукции – и является ключевым фактором конкурентоспособности предприятия [1-2].

Производственный цикл партии металла включает следующие этапы: поступление сырья и полупродуктов на переработку и входной контроль; металлургическую переработку (плавку, рафинирование, электролиз и иные технологические операции); лабораторные испытания, по результатам которых принимается решение о соответствии продукции нормативным требованиям; оформление паспорта качества и отгрузку (статус «Готовая продукция») либо направление на повторную переработку или в брак [3, 30].

Информационное обеспечение управления качеством металлопродукции строится на трёх ключевых классах систем.

Системы MES (Manufacturing Execution System) – системы управления производственными операциями – отвечают за управление производственными заданиями и диспетчеризацию, отслеживание хода выполнения технологических операций, мониторинг загрузки оборудования, управление персоналом и материалами, а также формирование производственной отчётности. В контексте контроля качества цветных металлов MES фиксирует параметры плавки, рафинирования, электролиза, режимы термообработки и иные технологические параметры, оказывающие непосредственное влияние на чистоту и свойства конечной продукции [5].

Системы SCADA (Supervisory Control and Data Acquisition) – системы диспетчерского управления и сбора данных – обеспечивают сбор сигналов с

датчиков и контрольно-измерительных приборов технологического оборудования в реальном времени, визуализацию состояния производственного процесса, автоматическое регулирование технологических параметров, а также регистрацию и архивирование показателей производственного процесса. SCADA предоставляет непрерывный поток данных об условиях производства: температуре расплавов, давлении, составе газовой среды, расходе реагентов и других параметрах технологических агрегатов [4].

Системы LIMS (Laboratory Information Management System) – лабораторные информационные системы – управляют потоком образцов в лаборатории: регистрацией поступающих образцов, назначением испытаний и исполнителей, вводом результатов измерений, верификацией данных, выдачей протоколов испытаний и удостоверений качества, а также хранением и статистической обработкой аналитических данных. LIMS хранит результаты химического анализа, физических испытаний и иных видов контроля качества металлопродукции [6].

Проблема взаимодействия между данными системами носит системный характер: каждая из них функционирует изолированно, использует собственные модели данных и форматы хранения, что существенно затрудняет получение консолидированного представления о качестве продукции. Для устранения этого разрыва необходима интеграционная платформа – «витрина качества металла», агрегирующая данные из всех трёх источников и предоставляющая их пользователям в удобном и структурированном виде в зависимости от их роли и потребностей.

1.2 Обзор существующих решений

1.2.1 Платформы сбора и аналитики промышленных данных

AVEVA PI System с модулем PI Vision – промышленная платформа агрегации и визуализации производственных данных компании AVEVA (ранее OSIsoft), являющаяся де-факто отраслевым стандартом для работы с

телеметрическими данными в металлургии, нефтегазовой и энергетической отраслях. Платформа состоит из нескольких компонентов: PI Data Archive – специализированный временной архив, оптимизированный для хранения миллионов показаний датчиков; PI Asset Framework – контекстуализация данных в иерархию активов предприятия; PI Vision – веб-интерфейс для создания и просмотра дашбордов с трендами. Система поддерживает подключение к тысячам типов промышленного оборудования через готовые коннекторы [7].

Достоинства AVEVA PI System: богатая экосистема интеграций с оборудованием SCADA и DCS-системами; мощные средства анализа временных рядов; высокая производительность при работе с большими объёмами телеметрии; широкое распространение в металлургической отрасли.

Недостатки: система ориентирована исключительно на работу с телеметрическими данными и не имеет нативной интеграции с LIMS; создание ролевых дашбордов требует ручной настройки PI Vision и не предоставляет готовой ролевой модели; отсутствует функциональность управления инцидентами и генерации PDF-отчётов по партиям; крайне высокая стоимость лицензирования.

dataPARC (PARCview) – промышленная аналитическая платформа, агрегирующая данные из MES, лабораторных систем и архивов SCADA в едином веб-интерфейсе. Предоставляет инструменты для анализа трендов производственных параметров, корреляционного анализа влияния технологических условий на качество продукции, а также настраиваемые дашборды для различных групп пользователей. Поддерживает импорт данных из лабораторных систем и исторических архивов [8].

Достоинства: универсальность подключения к разнородным источникам данных; наличие инструментов корреляционного анализа между параметрами процесса и качеством.

Недостатки: отсутствует встроенная ролевая модель доступа; нет

функционала отслеживания статусов производственных партий и их таймлайна; отсутствует управление реестром инцидентов; генерация PDF-отчётов по партиям не предусмотрена.

Таким образом, платформы сбора и аналитики промышленных данных обеспечивают высокую производительность при работе с телеметрическими потоками, однако не объединяют производственные и лабораторные данные в единой ролевой витрине и не поддерживают отслеживание жизненного цикла производственной партии.

1.2.2 Специализированные системы управления качеством металлургического производства

Primetals Technologies Quality System – специализированная система управления качеством металлургического производства, разработанная австрийским технологическим концерном Primetals Technologies. Система обеспечивает сквозное отслеживание качества продукции от плавки до финишной обработки, корреляцию технологических параметров с показателями качества, автоматическую классификацию дефектов и генерацию паспортов качества. Функционально охватывает данные из систем автоматизации Primetals, результаты лабораторных испытаний и параметры производственных операций [9].

Достоинства: глубокая специализация под задачи металлургического производства; полная трассируемость производственного цикла партии; интеграция данных от нескольких технологических переделов в одном интерфейсе.

Недостатки: система ориентирована на предприятия чёрной металлургии (прокатное производство) и не адаптирована к специфике производства цветных металлов; функционирует только в связке с оборудованием Primetals Technologies; высокая стоимость внедрения; отсутствуют ролевые витрины данных, настроенные под конкретные подразделения.

SMS group X-Pact Quality – система управления качеством металлургического производства от немецкого концерна SMS group, реализующая контроль качества по принципу «от плавки до рулона». Система обеспечивает автоматическую классификацию технологических отклонений, управление рекламациями, генерацию сертификатов качества и отслеживание производственных партий на всех переделах [10].

Достоинства: тесная интеграция с производственным оборудованием SMS group; поддержка международных стандартов качества; полная история производственного цикла партии.

Недостатки: применима только на предприятиях, оснащённых оборудованием SMS group; отсутствует интеграция с внешними LIMS-системами; нет ролевых аналитических витрин для производственного персонала и руководства; система ориентирована на стальной прокат.

Таким образом, специализированные металлургические системы управления качеством обладают высокой отраслевой экспертизой, однако ориентированы преимущественно на предприятия чёрной металлургии, жёстко привязаны к оборудованию конкретных производителей и не предоставляют персонализированных ролевых витрин данных.

1.2.3 Комплексные MES-платформы с модулем управления качеством

Siemens Opcenter Quality – специализированный модуль управления качеством в составе комплексной MES-платформы Siemens Opcenter. Функционал включает планирование инспекций и испытаний, статистическое управление процессами (SPC – Statistical Process Control) с автоматическим построением контрольных карт Шухарта, управление несоответствиями (NCR – Non-Conformance Reports), корректирующими и предупреждающими действиями (CAPA), а также интеграцию с лабораторными системами через стандартизированные интерфейсы. Платформа соответствует стандарту ISA-95 и поддерживает интеграцию с SCADA-системами Siemens [11].

Достоинства: наличие модуля SPC для статистического анализа процессных параметров; развитые инструменты управления несоответствиями; соответствие международным производственным стандартам; возможность интеграции MES и SCADA данных в единой платформе.

Недостатки: чрезвычайно высокая стоимость лицензирования и внедрения; требует привлечения авторизованных интеграторов Siemens; не предоставляет готовых ролевых витрин данных для разных производственных подразделений; ограниченная адаптируемость к специфике цветной металлургии.

Таким образом, комплексные MES-платформы с модулем управления качеством обеспечивают наиболее широкий функциональный охват среди рассмотренных решений, однако их высокая стоимость, сложность кастомизации и отсутствие готовой ролевой модели представления данных делают их применение нецелесообразным в контексте разрабатываемой системы.

1.3 Сравнительный анализ существующих решений

Для объективного обоснования необходимости разработки собственной информационной системы был проведён сравнительный анализ рассмотренных аналогов по ключевым функциональным и организационным критериям, непосредственно соответствующим требованиям к разрабатываемой системе. Результаты анализа представлены в таблице 1.

Как следует из приведённого ниже анализа, ни одно из рассмотренных решений не удовлетворяет в полной мере совокупности требований к разрабатываемой системе. Дополнительным ограничивающим фактором для всех зарубежных платформ является наличие санкционных рисков, делающих их применение в инфраструктуре Норникеля потенциально невозможным.

Таблица 1 – Сравнительный анализ существующих систем управления качеством металла

Критерий	AVEVA PI System	dataPARC	Primetals QS	SMS X-Pact	Siemens Opcenter Quality
Интеграция MES+SCADA+LIMS в единый интерфейс	Частично	Частично	Частично	Нет	Да
Ролевые витрины данных	Нет	Нет	Нет	Нет	Нет
Мониторинг отклонений в реальном времени	Да	Да	Да	Да	Да
Уведомления об отклонениях	Нет	Нет	Нет	Нет	Да
Управление реестром уведомлений о сбоях	Нет	Нет	Да	Да	Да
Автоматическая генерация PDF-отчётов по партии	Нет	Нет	Да	Да	Нет
Адаптация к цветной металлургии	Частично	Частично	Нет	Нет	Частично
Отсутствие санкционных рисков	Нет	Нет	Нет	Нет	Нет

Платформы AVEVA PI System и dataPARC обеспечивают высокую производительность при работе с телеметрическими данными, однако не интегрируют лабораторные данные LIMS и не предоставляют ролевых витрин. Специализированные металлургические системы Primetals и SMS X-Pact обладают необходимой производственной функциональностью, но ориентированы исключительно на предприятия чёрной металлургии и не поддерживают гибкую ролевую модель представления данных. Siemens Opcenter Quality – наиболее функционально полное решение из рассмотренных – требует значительных инвестиций в лицензирование и внедрение, не адаптировано к специфике производства цветных металлов и не предоставляет готовых персонализированных витрин для различных категорий производственного персонала.

Таким образом, ни одно из существующих решений не обеспечивает одновременно интеграцию MES, SCADA и LIMS в едином интерфейсе,

персонализированные ролевые витрины данных, управление реестром инцидентов с уведомлениями и генерацию отчётов по партиям, адаптированных к специфике производства цветных металлов. Именно этим определяется необходимость разработки специализированной информационной системы витрины качества металла.

1.4 Постановка задачи на разработку

На основании проведённого анализа предметной области, существующих корпоративных решений в области управления качеством металлопродукции, а также особенностей организации производственных процессов на предприятиях металлургической отрасли была сформулирована задача на разработку информационной системы витрины качества металла. Основной целью разрабатываемой системы является централизованное хранение, обработка и визуализация производственных и лабораторных данных, поступающих из различных источников, а также обеспечение оперативного контроля качества металлопродукции на различных этапах производственного цикла.

Разрабатываемая система должна обеспечивать сбор, обработку и хранение данных о партиях металла из трёх внешних источников: MES-систем, содержащих сведения о технологических параметрах переработки металлов; SCADA-систем, предоставляющих телеметрические данные и показатели технологического оборудования в режиме реального времени; а также LIMS-систем, обеспечивающих хранение испытаний продукции. Для каждой производственной партии система должна поддерживать возможность отслеживания её текущего состояния, отображения производственных параметров и анализа отклонений от нормативных показателей.

Важным функциональным требованием является реализация ролевой модели доступа пользователей. Система должна поддерживать роли лаборатории, производства, руководства и администратора, каждая из которых получает доступ к специализированной витрине данных,

соответствующей интересам конкретной категории пользователей. Сотрудники лаборатории должны иметь доступ преимущественно к результатам исследований LIMS, производственный персонал – к данным MES и SCADA, руководство – к агрегированным аналитическим показателям, а администраторы – к средствам управления пользователями и ролями доступа.

Дополнительно система должна поддерживать автоматическое формирование уведомлений при выходе производственных параметров за допустимые пределы, генерацию PDF-отчётов по партиям металла.

К числу нефункциональных требований относятся обеспечение надёжности и отказоустойчивости системы, поддержка ролевого разграничения доступа, обеспечение высокой скорости отображения данных и передачи уведомлений, а также создание интуитивно понятного пользовательского интерфейса, адаптированного для работы производственного персонала. Дополнительным требованием является возможность контейнеризованного развёртывания системы и поддержка масштабируемой архитектуры, ориентированной на обработку потоковых производственных данных.

Таким образом, в результате анализа предметной области и формирования постановки задачи были определены основные функциональные и нефункциональные требования к информационной системе витрины качества металла. Полученные результаты служат основой для последующего проектирования структуры системы и выбора технологий её реализации.

2 ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ

2.1 Функциональные возможности системы

Витрина качества металла обладает следующим функционалом, соответствующим сформулированным требованиям на разработку:

- Ролевая аутентификация и разграничение доступа с персонализированными витринами данных в соответствии с ролью пользователя;
- Мониторинг партий металла в режиме реального времени с отображением агрегированных счётчиков по всем статусам жизненного цикла в разрезе каждого типа металла;
- Трассируемость производственного цикла партии посредством хронологического таймлайна смены статусов с временными метками каждого перехода;
- Визуализация технологических параметров в виде интерактивных графиков временных рядов показаний датчиков SCADA для каждой партии с подсветкой выявленных отклонений;
- Автоматическое обнаружение отклонений показателей SCADA от нормативных порогов при поступлении данных с формированием инцидента и мгновенной доставкой уведомления ответственным пользователям;
- Генерация PDF-отчёта по партии металла с автоматическим сбором данных MES, SCADA и LIMS, подсчётом количества отклонений от нормы по каждому источнику и формированием итогового заключения о качестве партии;
- Управление пользователями и ролями в административном разделе системы.

2.2 Описание данных системы

В отличие от традиционных корпоративных информационных систем, ориентированных преимущественно на работу со статическими данными,

проектируемая система обрабатывает как структурированные справочные сведения, так и потоковые телеметрические данные, поступающие в режиме, близком к реальному времени. Значительная часть информации характеризуется высокой частотой обновления, что предъявляет повышенные требования к скорости обработки, производительности базы данных и эффективности механизмов передачи сообщений.

2.2.1 Входные данные

К основным входным данным системы относятся сведения о партиях из сервиса-эмулятора, а также данные пользователей и параметры взаимодействия компонентов системы, что можно увидеть в таблице 2.

Таблица 2 – Основные входные данные системы

Данные	Описание
Данные MES	Производственные параметры партии металла
Данные SCADA	Показатели датчиков и оборудования
Данные LIMS	Результаты лабораторных анализов
Логин + пароль / JWT-токен	Данные аутентификации и авторизации пользователя

Производственные данные MES содержат сведения о партиях металла: номере производственного заказа, идентификаторах партии, оборудования и оператора, на каком этапе производства находится партия, временные отметки смены статусов, продолжительность обработки, весовые показатели.

Данные SCADA характеризуются высокой интенсивностью поступления и содержат идентификаторы датчика и оборудования, временную метку измерений, параметр технологического оборудования и его значение на момент измерения и информацию о соответствии значения норме.

Лабораторные данные LIMS включают результаты исследований образцов металла, сведения о применяемых методах анализа, нормативных значениях показателей и итоговых заключениях лаборатории.

Помимо производственной информации, система принимает данные

пользователей: либо введенные логин и пароль при первом обращении к системе, либо JWT-токен в последующих запросах, необходимые для работы механизмов безопасности и маршрутизации.

Таким образом, входные данные системы представляют собой совокупность производственной, лабораторной, телеметрической и служебной информации, поступающей из различных источников в асинхронном режиме.

2.2.2 Выходные данные

Результатом работы информационной системы являются данные, отображаемые пользователям в интерфейсе витрины качества металла, а также отчётные материалы, формируемые сервером. Все категории выходных данных можно рассмотреть в таблице 3.

Таблица 3 – Основные выходные данные системы

Данные	Описание
Уведомления	Сообщения о технологических отклонениях
Информация о партии	Сведения о состоянии и параметрах партии металла
PDF-отчёт	Сводный отчёт по производственной партии

В пользовательском интерфейсе данные организуются в зависимости от роли пользователя. Производственный персонал получает доступ к информации о текущем состоянии партий металла, технологических параметрах и уведомлениях об отклонениях. Сотрудники лаборатории работают преимущественно с результатами исследований и показателями качества продукции. Руководство предприятия получает агрегированные аналитические показатели, статистику брака, позволяющие оценивать эффективность производственного процесса и уровень качества металлопродукции, а также имеют доступ к уведомлениям о сбоях.

Особое значение имеют уведомления, формируемые системой при обнаружении отклонений технологических параметров от нормативных значений. Такие уведомления передаются пользователям посредством

WebSocket-соединений и отображаются в интерфейсе в режиме реального времени.

Отдельным видом выходных данных являются PDF-отчёты по партиям металла. Генерируемые документы содержат сведения о производственных параметрах, показателях SCADA, результатах лабораторных исследований LIMS, зарегистрированных отклонениях и итоговом статусе партии. Подобные отчёты могут использоваться для внутреннего документооборота.

Таким образом, выходные данные информационной системы обеспечивают поддержку оперативного мониторинга, аналитической обработки и формирования отчётности по производственным партиям металла.

2.3 Описание процессов и потоков системы

Проектирование информационной системы предполагает формализованное описание составляющих её процессов и информационных потоков с использованием стандартизированных нотаций моделирования. Применение различных методологий позволяет рассмотреть систему с дополняющих друг друга точек зрения: функциональной, процессной, потоковой и структурной. В данном разделе представлены модели, разработанные с использованием методологий IDEF0 и IDEF3, а также диаграммы в нотации UML (диаграмма прецедентов, диаграмма активности, диаграмма последовательности).

2.3.1 IDEF0

Контекстная диаграмма IDEF0 (рис. 1) описывает процесс обеспечения контроля качества партии металла [12-13]. Входными данными функции служат производственные параметры из систем MES, показатели датчиков из SCADA и результаты лабораторных анализов из LIMS, а также учётные данные пользователя для аутентификации. Управление осуществляется нормативными значениями показателей качества для каждого типа металла, требованиями стандартов ISO 9001 и ISO/IEC 17025 к управлению данными

качества, а также ролевой политикой разграничения доступа (RBAC). Выходами функции являются ролевая витрина данных, отображаемая пользователю в соответствии с его производственной ролью; уведомления об обнаруженных отклонениях; PDF-отчёт по производственной партии. Механизмами реализации выступают компоненты информационной системы и запросы пользователя к системе.

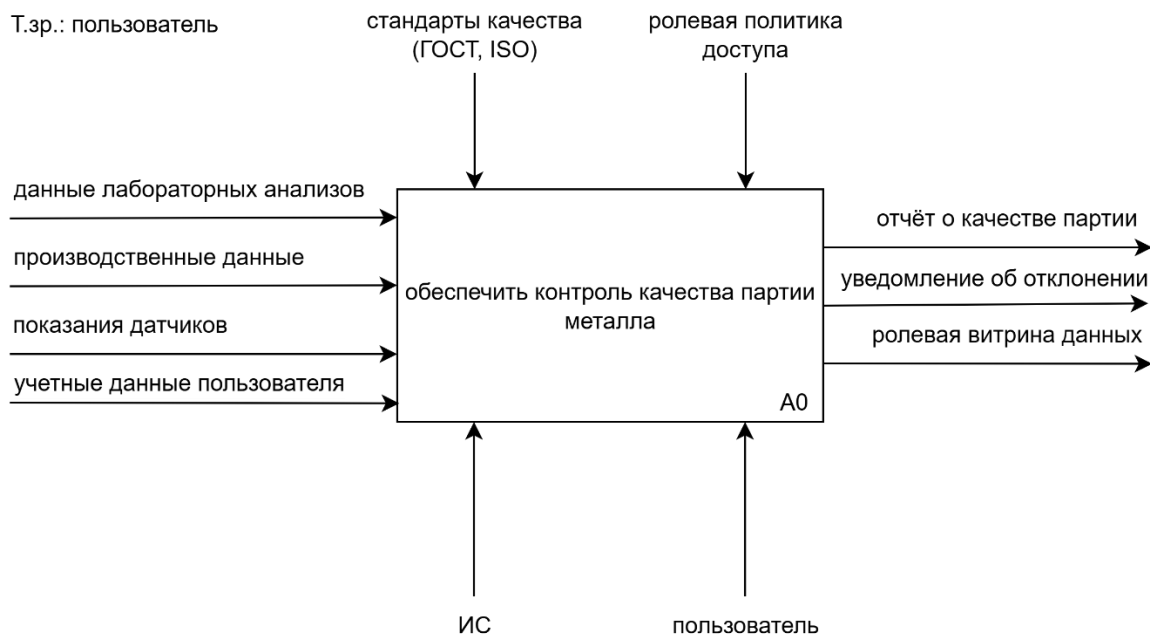


Рисунок 1 – Диаграмма IDEF0

Декомпозиция диаграммы IDEF0 (рис. 2) раскрывает внутреннюю функциональную структуру процесса, состоящую из четырёх взаимосвязанных подфункций. Подфункция A1 «получить из источников и сохранить данные о партии» принимает сообщения из топиков Apache Kafka, выполняет их десериализацию и валидацию, после чего сохраняет данные в соответствующие таблицы базы данных. Результат выполнения A01 – сохранённые и нормализованные данные партии – передаётся в подфункцию A02. Подфункция A02 «проверить показатели на признак отклонений» определяет статус поступивших значений параметров и формирует запись инцидента при обнаружении превышения нормы и инициирует отправку уведомления. Подфункция A03 «предоставить ролевую витрину данных»

обрабатывает запросы авторизованных пользователей: в зависимости от роли запрашивает из базы данных соответствующий набор сведений. Подфункция A04 «сформировать отчёт» по запросу пользователя собирает данные из таблиц БД по всем источникам для указанной партии, подсчитывает количество зафиксированных отклонений и формирует структурированный PDF-документ.

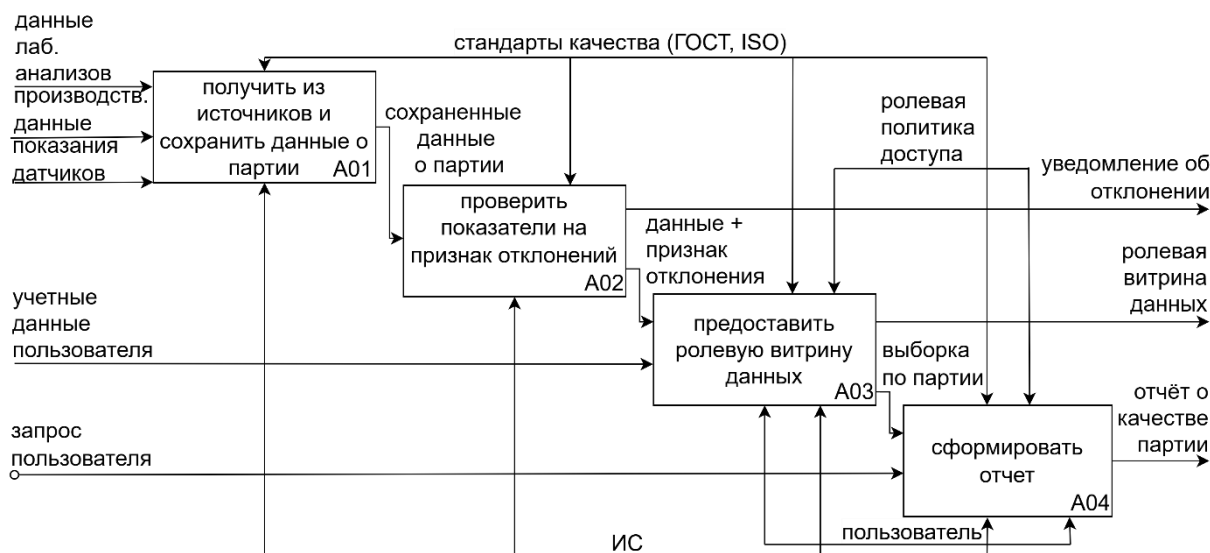


Рисунок 2 – Декомпозиция диаграммы IDEF0

Таким образом, функциональная модель IDEF0 демонстрирует, что система витрины качества металла реализует цепочку обработки производственных данных – от получения из внешних источников до представления пользователю в персонализированном интерфейсе и документирования в форме отчётов.

2.3.2 IDEF3

В рамках системы в нотации IDEF3 смоделирован процесс обработки инцидента – от момента поступления данных с отклонением до закрытия зарегистрированного инцидента ответственным пользователем [12-13]. Диаграмма изображена на рисунке 3.

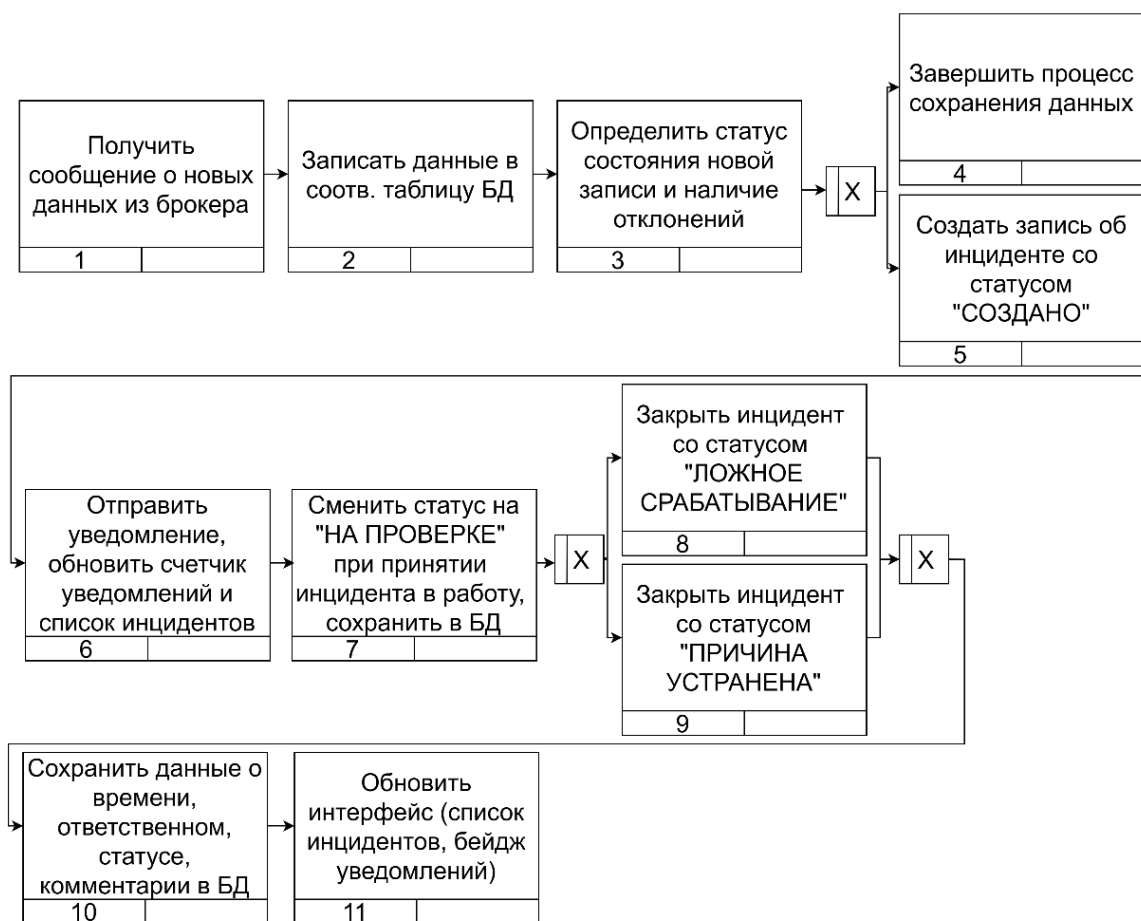


Рисунок 3 – Диаграмма IDEF3

Процесс инициируется в момент получения серверным приложением сообщения из брокера. Сервер десериализует входящее JSON-сообщение и производит запись новых данных в соответствующую таблицу БД. После сохранения данных определяется статус соответствия значений норме. Если статус показывает выход за допустимые пределы, создаётся новая запись инцидента со статусом «Создано», и через WebSocket-канал пользователям с ролями «Производство» и «Руководство», подключённым к системе в данный момент, направляется уведомление. Пользователь получает уведомление в интерфейсе, переходит на страницу списка сбоев, просматривает детали уведомления и принимает инцидент в работу, переводя его в статус «На проверке». После проведения необходимых корректирующих мероприятий на производстве пользователь закрывает инцидент и в зависимости от истинности сбоя устанавливает статус «Причина устранена» или «Ложное

срабатывание».

Таким образом, диаграмма IDEF3 позволяет формализовать последовательность реакции системы на производственные отклонения – от автоматического обнаружения на сервере до подтверждения устранения пользователем, – что обеспечивает полную прослеживаемость каждого сбоя в жизненном цикле производственной партии.

2.3.3 UML

В данном разделе применены несколько видов UML-диаграмм, каждая из которых моделирует отдельный аспект системы [14].

Диаграмма прецедентов (Use Case Diagram) системы витрины качества металла (рис. 4) устанавливает четыре актора: Лаборатория, Производство, Руководство, Администратор.

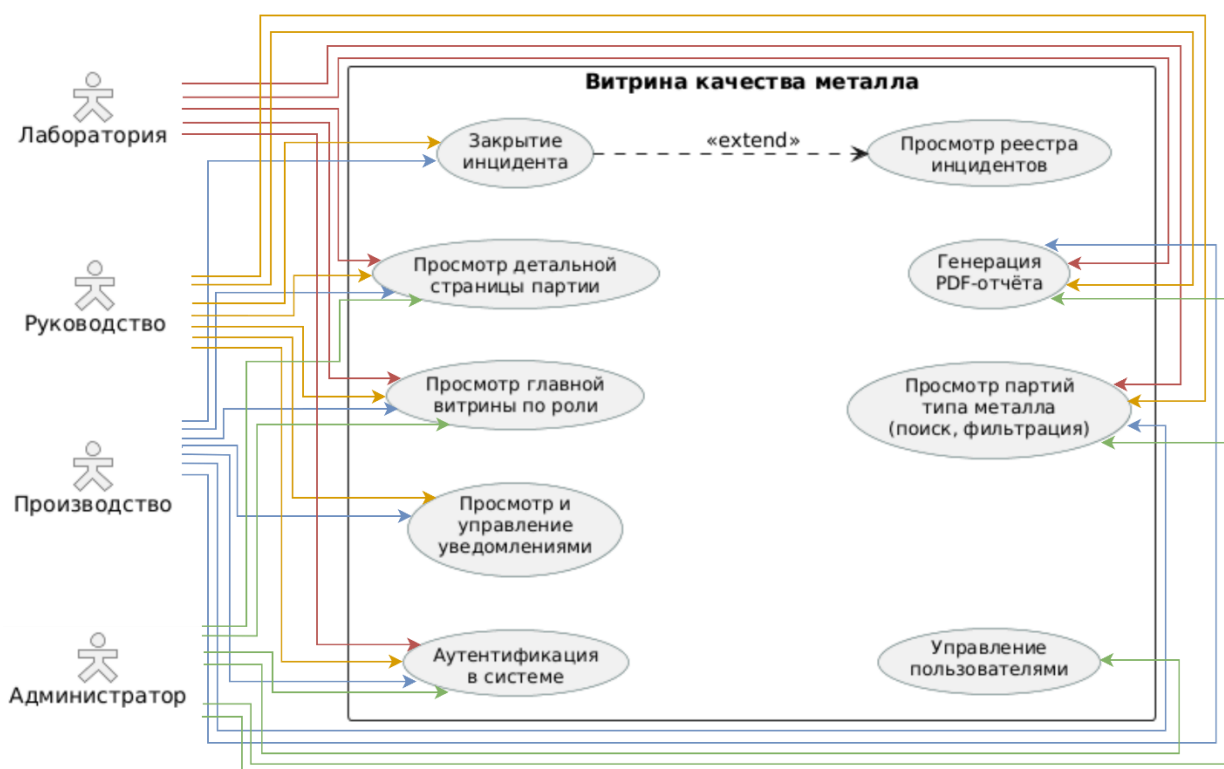


Рисунок 4 – Диаграмма прецедентов

Каждый из акторов (ролей в системе) имеет доступ к персонализированной витрине, списку партий с фильтрацией, детальной странице партии и генерации PDF-отчёта. Ролям «Производство» и «Руководство» также доступно управление уведомлениями о сбоях и

просмотр страницы списка инцидентов. А роль «Администратор» дополнительно имеет возможность управления пользователями.

Диаграмма активности (Activity Diagram), изображенная на рисунке 5, описывает процесс генерации PDF-отчёта по производственной партии металла.

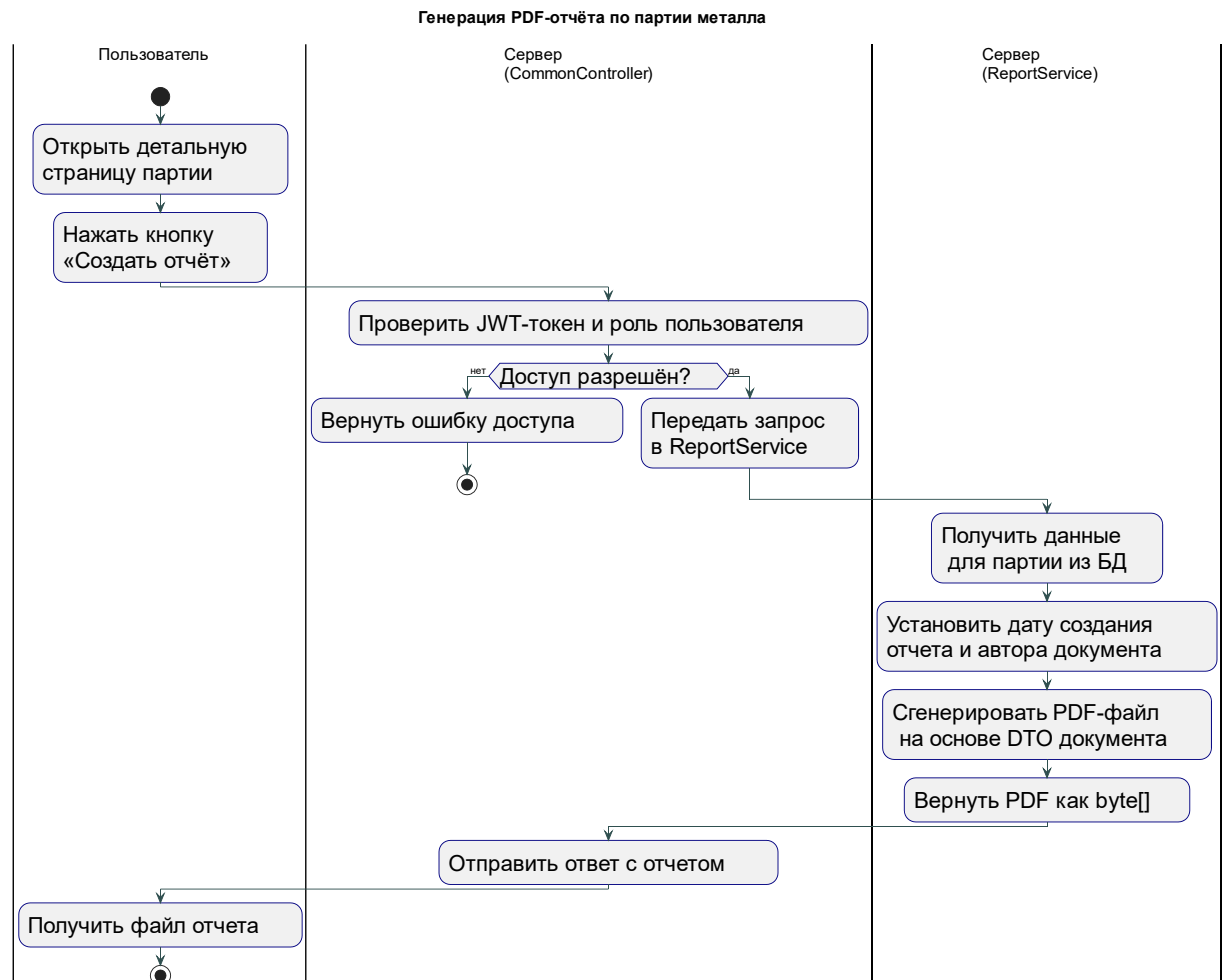


Рисунок 5 – Диаграмма активности

Процесс инициируется нажатием пользователем кнопки «Создать отчёт» на детальной странице партии, после чего клиентское приложение отправляет запрос на сервер. Серверное приложение проверяет подлинность токена в запросе, после чего запрашивает из БД необходимый набор данных. Далее формирует PDF-документ, содержащий всю необходимую информацию и итоговое заключение о качестве партии. Готовый документ возвращается клиенту и открывается в новом окне браузера.

На диаграмме последовательности (Sequence Diagram), изображенной на рисунке 6, представлен сценарий взаимодействия пользователя, клиентского приложения, серверного приложения и базы данных. Диаграмма охватывает три последовательных сценария: аутентификацию, получение информации о партии и генерацию отчёта.

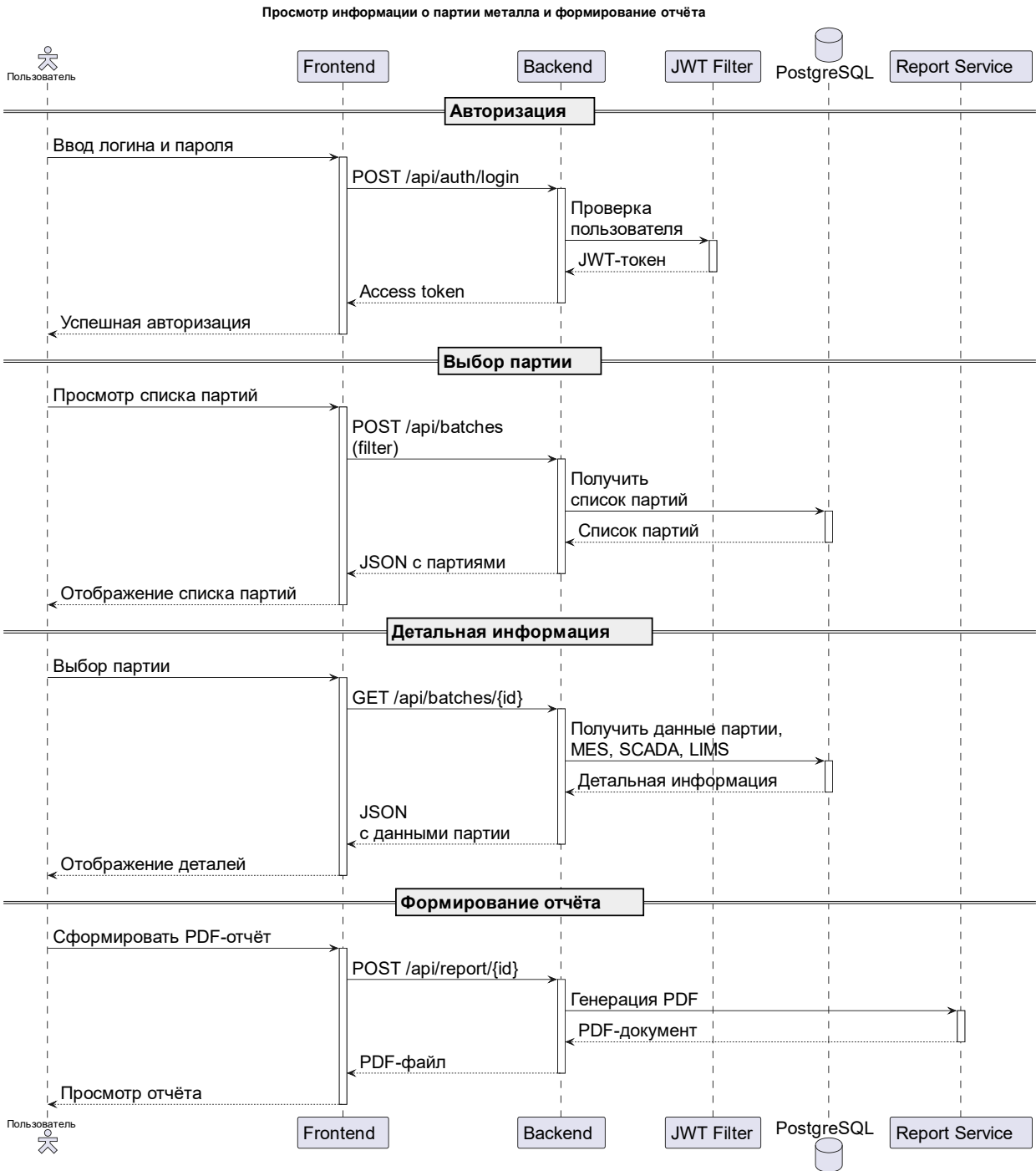


Рисунок 6 – Диаграмма последовательности

Таким образом, структура системы, информационные процессы и

потоки, описанные с помощью нотации UML, обеспечивают отслеживание производственных данных на всех этапах их жизненного цикла.

2.4 Архитектура системы

Архитектура разрабатываемой информационной системы построена по многоуровневой клиент-серверной модели и включает клиентское приложение, серверное приложение, сервис-эмулятор производственных данных, брокер сообщений и реляционную базу данных. Подобная структура обеспечивает разделение функций между компонентами системы и позволяет организовать централизованную обработку производственной информации.

Клиентское приложение предназначено для взаимодействия пользователей с системой и обеспечивает отображение ролевых витрин данных, информации о партиях металла, результатов лабораторных исследований, производственных параметров, аналитических графиков и уведомлений о технологических отклонениях. Интерфейс также предоставляет функции поиска партий, фильтрации данных и формирования отчётов.

Серверное приложение выполняет функции централизованной обработки бизнес-логики системы. Оно принимает запросы от клиентского приложения, обрабатывает производственные данные, управляет пользователями и ролями доступа, формирует уведомления, агрегирует сведения из источников и подготавливает PDF-отчёты по производственным партиям.

Сервис-эмулятор производственных данных предназначен для имитации работы внешних систем MES, SCADA и LIMS. Он формирует тестовые потоки производственной, телеметрической и лабораторной информации и передаёт их в брокер сообщений, обеспечивая моделирование поступления данных в систему в режиме, близком к реальному времени.

Брокер сообщений обеспечивает передачу данных между сервисом-эмулятором и серверным приложением. Использование брокера позволяет

организовать асинхронный обмен сообщениями, разделить потоки данных по категориям и обеспечить устойчивую обработку информации при высокой интенсивности поступления сообщений.

База данных используется для хранения информации о партиях металла, технологических параметрах, результатах лабораторных исследований, уведомлениях и учётных записях пользователей. Хранилище данных обеспечивает сохранение истории производственных операций и поддержку аналитической обработки информации.

Взаимодействие основных компонентов системы представлено на рисунке 7.

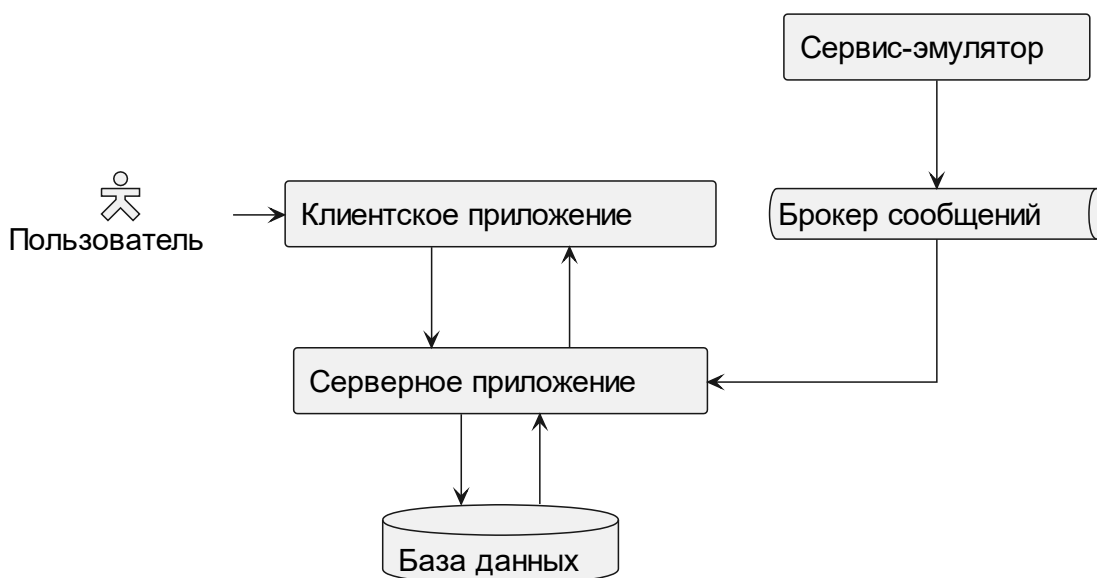


Рисунок 7 – Диаграмма архитектуры системы

Разработанная архитектура информационной системы обеспечивает эффективное взаимодействие между компонентами программного комплекса, поддержку потоковой обработки производственных данных и централизованное хранение информации.

3 ОПИСАНИЕ ПРАКТИЧЕСКОЙ РЕАЛИЗАЦИИ СИСТЕМЫ

3.1 Выбор технологий реализации

3.1.1 Технологии разработки клиентской части

Библиотека React 19 выбрана в качестве основы клиентского приложения по двум причинам [15]. Во-первых, механизм Virtual DOM обеспечивает обновление только тех компонентов интерфейса, данные которых изменились, так как перерисовка всего DOM-дерева при каждом обновлении делала бы интерфейс неотзывчивым. Во-вторых, компонентная архитектура позволила создать переиспользуемые блоки, которые применяются без дублирования кода во всех четырёх ролевых витринах.

Для организации маршрутизации применяется библиотека React Router [16]. Использование данного решения обеспечивает навигацию между разделами системы без полной перезагрузки страницы и позволяет реализовать механизм защищённых маршрутов через компонент ProtectedRoute.

Одним из ключевых компонентов клиентской архитектуры является библиотека React Query, предназначенная для управления состоянием, кэширования данных и синхронизации интерфейса с серверным приложением. Использование React Query позволяет снизить количество повторных обращений к серверу, ускорить обновление интерфейса и повысить отзывчивость системы при работе с производственными данными [17].

Библиотека Axios выбрана из-за механизма перехватчиков (interceptors) [18]. Request interceptor автоматически извлекает JWT-токен из localStorage и добавляет его в заголовок Authorization каждого исходящего запроса, исключая необходимость вручную передавать токен в каждом вызове API. Response interceptor перехватывает ответы с определенными кодами, обеспечивая единообразную обработку ошибок по всему приложению.

Для визуализации производственных данных используется библиотека Recharts. Применение данного решения обусловлено необходимостью

построения интерактивных графиков, отображающих динамику изменения технологических параметров без разработки собственного графического движка [19].

3.1.2 Технологии разработки серверной части

Серверная часть системы реализована на языке Java 21 с использованием фреймворка Spring Boot 3. Выбор Java обусловлен высокой надёжностью платформы, поддержкой многопоточности и широким распространением технологии в корпоративной разработке. Использование Spring Boot позволяет упростить конфигурацию серверного приложения, ускорить разработку REST API и обеспечить интеграцию с другими компонентами экосистемы Spring [20].

Spring Security + JWT: JWT-фильтр интегрирован в цепочку безопасности Spring Security как первый обработчик (OncePerRequestFilter) и извлекает идентификатор пользователя и его роль из тела токена без обращения к базе данных при каждом запросе. Это обеспечивает stateless-аутентификацию: поскольку сервер не хранит пользовательских сессий, любой из нескольких экземпляров серверного приложения может обработать любой запрос, что является необходимым условием горизонтального масштабирования [21-22].

Для взаимодействия с базой данных применяется Spring Data JPA совместно с ORM-фреймворком Hibernate. Использование JPA позволяет реализовать объектно-реляционное отображение сущностей и сократить объём низкоуровневого SQL-кода. Hibernate обеспечивает автоматическое управление жизненным циклом сущностей и оптимизацию работы с базой данных [23-24].

iText 5 выбрана для серверной генерации PDF-отчётов, поскольку предоставляет методы для построения документов с табличным представлением данных, встраиванием кириллических TTF-шрифтов и управлением разбивкой больших таблиц на страницы [25].

3.1.3 База данных

В качестве системы управления базами данных выбрана PostgreSQL 18. Данная СУБД отличается высокой надёжностью, поддержкой сложных аналитических запросов и возможностью эффективной работы с большими объёмами данных. Также она обладает механизмами индексации и партиционирования, необходимыми для быстрого доступа к определенным производственным данным [26].

3.1.4 Взаимодействие сервисов

Apache Kafka используется для асинхронного обмена информацией между сервисом-эмулятором и серверным приложением. Разделение потоков по топикам брокера позволяет серверу обрабатывать каждый тип данных с независимой скоростью. Журнальное хранение сообщений (log retention) обеспечивает гарантию доставки: если серверное приложение было перезапущено, все накопившиеся в топике сообщения будут обработаны после восстановления, без потери данных о производственных операциях или показаниях датчиков [27].

WebSocket (Spring WebSocket) выбран как механизм доставки уведомлений и новых данных в клиентское приложение. Использование постоянного двустороннего соединения позволяет мгновенно передавать информацию пользователям системы без необходимости периодического опроса сервера [28].

Docker и Docker Compose применяются для контейнеризации и оркестрации сервисов системы. Docker обеспечивает изоляцию каждого сервиса, исключая конфликты версий зависимостей. Docker Compose позволяет декларативно задать порядок запуска сервисов через директиву `depends_on` и передать конфигурацию через переменные окружения без изменения исходного кода, что обеспечивает развёртывание всего комплекса единственной командой [29].

Таким образом, каждый компонент технологического стека был выбран

на основании конкретных функциональных характеристик, непосредственно обеспечивающих ключевые требования к системе.

3.2 Реализация базы данных

Проектирование базы данных являлось одним из ключевых этапов разработки. Основной задачей было создание структуры хранения данных, обеспечивающей высокую производительность при работе с большими объемами технологической информации.

3.2.1 Реализация таблиц

Таблица dim_batch хранит основные данные о производственном цикле партии, полученные из системы MES. Информацию о ее структуре можно увидеть в таблице 4. Данная таблица БД выступает в роли основной сущности, с которой связаны производственные (fact_mes), лабораторные (fact_lims) и телеметрические данные (fact_scada).

Таблица 4 – Структура таблицы dim_batch

№	Поле	Тип данных	Ограничение	Назначение
1	batch_id	TEXT	PRIMARY KEY	Идентификатор партии
2	metal_type	TEXT	NOT NULL	Тип производимого металла
3	start_time	TIMESTAMP	—	Время поступления партии
4	processing_time	TIMESTAMP	—	Время начала обработки
5	analyses_time	TIMESTAMP	—	Время завершения обработки и начала лабораторных анализов
6	end_time	TIMESTAMP	—	Время завершения анализов и производственного цикла
7	process_status	TEXT	—	Текущий статус партии
8	output_yield	DOUBLE PRECISION	—	Выход годной продукции

Для хранения технологических параметров, поступающих из MES, используется таблица fact_mes. Ее структура отражена в таблице 5.

Таблица 5 – Структура таблицы fact_mes

№	Поле	Тип данных	Ограничение	Назначение
1	record_id	TEXT	PRIMARY KEY	Уникальный идентификатор записи
2	order_id	TEXT	NOT NULL	Идентификатор производственного заказа
3	batch_id	TEXT	NOT NULL, FK	Идентификатор партии
4	equipment_id	TEXT	NOT NULL	Идентификатор оборудования, в котором производится обработка
5	operator_id	TEXT	—	Идентификатор оператора, который зарегистрировал обработку
6	charge_mass	DOUBLE PRECISION	—	Масса исходной шихты, т
7	output_mass	DOUBLE PRECISION	—	Масса готовой продукции, т
8	duration_min	INT	—	Продолжительность технологической обработки, мин

Данные лабораторных анализов, поступающие из LIMS, размещаются в таблицах fact_lims и fact_lims_results. Таблица fact_lims предназначена для хранения общей информации о лабораторных исследованиях. А детализированные результаты измерений сохраняются в таблице fact_lims_results. Их структура отражена в таблицах 6 и 7.

Таблица 6 – Структура таблицы fact_lims

№	Поле	Тип данных	Ограничение	Назначение
1	record_id	TEXT	PRIMARY KEY	Уникальный идентификатор анализа
2	batch_id	TEXT	NOT NULL, FK	Идентификатор партии
3	sample_id	TEXT	NOT NULL	Идентификатор исследуемой пробы
4	analysis_method	TEXT	—	Метод проведения анализа
5	test_date	TIMESTAMP	—	Дата и время проведения анализа
6	status	TEXT	—	Статус итога исследования

Таблица 7 – Структура таблицы fact_lims_results

№	Поле	Тип данных	Ограничение	Назначение
1	id	SERIAL	PRIMARY KEY	Уникальный идентификатор результата
2	record_id	TEXT	NOT NULL, FK	Идентификатор лабораторного анализа
3	parameter_name	TEXT	—	Наименование исследуемого параметра
4	value	TEXT	—	Полученное значение параметра
5	unit	TEXT	—	Единица измерения
6	normal	BOOLEAN	DEFAULT TRUE	Признак соответствия нормативу

Для хранения данных SCADA применяется таблица fact_scada, структуру которой можно увидеть в таблице 8.

Таблица 8 – Структура таблицы fact_scada

№	Поле	Тип данных	Ограничение	Назначение
1	record_id	TEXT	NOT NULL, PK	Идентификатор записи
2	sensor_id	TEXT	NOT NULL	Идентификатор датчика
3	equipment_id	TEXT	NOT NULL	Идентификатор оборудования
4	time	TIMESTAMP	NOT NULL, PK	Время регистрации значения
5	parameter	TEXT	—	Технологический параметр
6	value	DOUBLE PRECISION	—	Значение параметра
7	unit	TEXT	—	Единица измерения
8	status	TEXT	—	Статус соответствия норме

Информация о зарегистрированных уведомлениях о сбоях хранится в таблице fact_notifications. Фиксирующиеся в ней данные отражены в таблице 9.

Таблица 9 – Структура таблицы fact_notifications

№	Поле	Тип данных	Ограничение	Назначение
1	id	SERIAL	PRIMARY KEY	Идентификатор уведомления
2	message	TEXT	—	Текст уведомления
3	equipment_id	TEXT	—	Идентификатор оборудования
4	sensor_id	TEXT	—	Идентификатор датчика
5	signal_source	TEXT	—	Источник сигнала
6	severity	TEXT	—	Уровень критичности
7	status	TEXT	—	Статус обработки уведомления
8	comment	TEXT	—	Комментарий пользователя
9	viewed	BOOLEAN	DEFAULT FALSE	Признак отправки на проверку
10	created_at	TIMESTAMP	DEFAULT NOW()	Дата и время создания уведомления
11	updated_at	TIMESTAMP	—	Дата и время последнего изменения уведомления
12	updated_by	TEXT	—	Пользователь, выполнивший изменение

Сведения о пользователях системы и их ролях размещаются в таблице users, поля которой можно рассмотреть в таблице 10. Данная таблица используется механизмами Spring Security и JWT-аутентификации для реализации разграничения доступа к интерфейсам и API системы в соответствии с ролевой моделью предприятия.

Таблица 10 – Структура таблицы users

№	Поле	Тип данных	Ограничение	Назначение
1	user_id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Уникальный идентификатор пользователя
2	username	TEXT	UNIQUE, NOT NULL	Логин пользователя
3	password	TEXT	NOT NULL	Хэш пароля пользователя
4	name	TEXT	—	Имя пользователя
5	surname	TEXT	—	Фамилия пользователя
6	patronymic	TEXT	—	Отчество пользователя
7	email	TEXT	—	Адрес электронной почты
8	role	TEXT	NOT NULL	Роль пользователя в системе
9	created_at	TIMESTAMP	DEFAULT NOW()	Дата и время регистрации пользователя

В совокупности таблицы dim_batch, fact_mes, fact_lims, fact_lims_results и fact_scada реализуют модель данных производственной партии металла, обеспечивающую трассируемость партии для пользователей. ER-диаграмма базы данных приведена на рисунке 8.

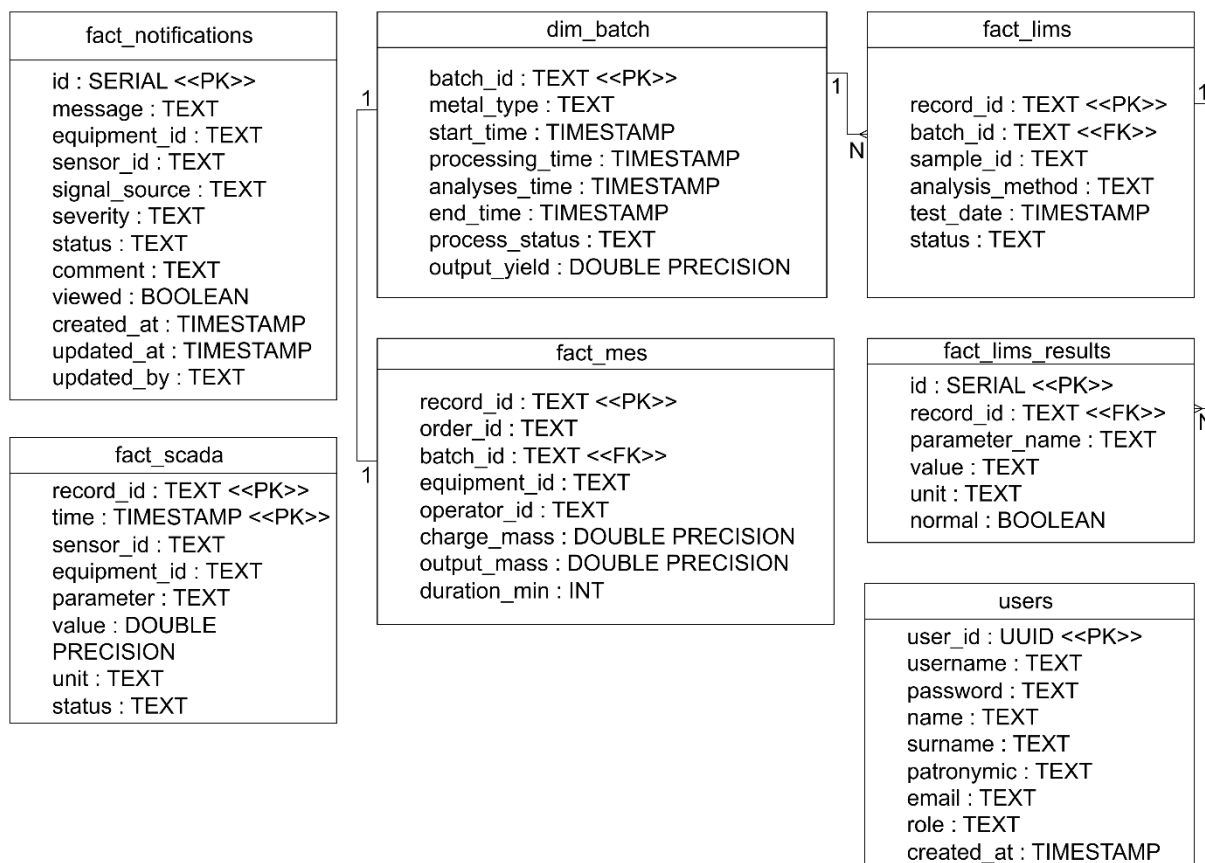


Рисунок 8 – ER-диаграмма базы данных

Таким образом, в рамках разработки информационной системы была спроектирована реляционная база данных, обеспечивающая централизованное хранение информации о производственных партиях, уведомлениях об отклонениях и пользователях системы.

3.2.2 Реализация индексации и партиционирования

С целью повышения производительности аналитических запросов и ускорения обработки больших объёмов производственных данных в базе данных реализована система индексов.

Для таблиц, содержащих данные о партиях, лабораторных анализах, уведомлениях и показаниях оборудования, созданы B-tree индексы, являющиеся стандартным и наиболее универсальным типом индексов в PostgreSQL. Данные индексы используются для ускорения поиска по идентификаторам партий, оборудования, датам проведения анализов, статусам технологических процессов и другим атрибутам, активно

участвующим в фильтрации и сортировке данных. Для некоторых сценариев дополнительно реализованы составные индексы, включающие сразу несколько полей. Такой подход позволяет эффективно выполнять запросы, использующие одновременно несколько условий отбора.

Поскольку таблица хранения данных SCADA содержит большой объем временных рядов, для неё дополнительно используется индекс типа BRIN (Block Range Index). Данный тип индекса оптимизирован для работы с последовательно поступающими временными данными и обеспечивает эффективный поиск записей по временным диапазонам при минимальном объеме занимаемой памяти.

Для обеспечения дальнейшей масштабируемости системы таблица `fact_scada` реализована с использованием механизма секционирования (партиционирования) по диапазонам времени. В качестве ключа партиционирования выбран временной атрибут записи (`time`). Каждая секция хранит данные только за один календарный месяц, что позволяет существенно уменьшить объем данных, обрабатываемых при выполнении запросов за ограниченный временной интервал. Дополнительно создана секция по умолчанию, предназначенная для хранения записей, не попавших ни в один из существующих временных диапазонов. Это обеспечивает корректное сохранение данных даже при возникновении нештатных ситуаций, связанных с отсутствием необходимой партии.

Для автоматизации сопровождения структуры хранения разработана специальная функция базы данных – `create_next_scada_partition`, выполняющая создание новой месячной партии для таблицы данных SCADA. Функция определяет границы следующего календарного месяца, проверяет наличие соответствующей секции и при её отсутствии автоматически создаёт новую партию с необходимым временным диапазоном. Автоматический запуск функции реализован с использованием расширения `pg_cron`. Планировщик выполняет проверку и создание новой

партиции 25-ого числа каждого месяца, что позволяет заранее подготовить структуру хранения к поступлению новых данных без участия администратора базы данных. Такой подход обеспечивает непрерывную работу системы и исключает риск возникновения ошибок при записи данных в начале нового отчетного периода.

Таким образом, разработанная структура базы данных обеспечивает централизованное хранение производственной, лабораторной и телеметрической информации, поддерживает ролевую модель доступа и позволяет эффективно выполнять аналитическую обработку данных.

3.3 Реализация взаимодействия компонентов системы

3.3.1 Брокер сообщений

Для организации обмена данными между сервисом-эмулятором производственных источников информации и серверной частью системы используется брокер сообщений Apache Kafka.

В разработанной системе создано три специализированных топика Kafka, соответствующих основным источникам производственных данных: MES, SCADA и LIMS. На стороне серверного приложения реализованы специализированные потребители сообщений (KafkaConsumerMes, KafkaConsumerScada, KafkaConsumerLims), осуществляющие чтение данных из соответствующих топиков. После получения сообщения выполняется его преобразование в объект предметной области, валидация и сохранение в базе данных. Такой подход обеспечивает централизованную обработку данных независимо от источника их происхождения.

3.3.2 API

Взаимодействие между клиентской и серверной частями системы реализовано посредством REST API. Данный подход обеспечивает стандартизированный обмен данными по протоколу HTTP.

Все программные интерфейсы объединены в единую структуру маршрутов и предоставляют доступ к функциональным возможностям

системы в соответствии с ролью пользователя. Перед выполнением большинства запросов осуществляется проверка подлинности пользователя на основе JWT-токенов. После успешной авторизации токен передается клиентом в заголовке каждого последующего запроса, что позволяет серверу идентифицировать пользователя и определить его права доступа.

Список основных маршрутов можно увидеть в таблице 11.

Таблица 11 – Основные методы API

Метод	Маршрут	Назначение
POST	/api/auth/login	Аутентификация пользователя
POST	/api/auth/register	Регистрация пользователя
POST	/api/metals	Получение списка партий металла с фильтром
GET	/api/batches/{id}	Получение детальной информации о партии
POST	/api/notifications	Получение всех уведомлений с фильтром
POST	/api/report/{id}	Генерация PDF-отчёта о партии
POST	/api/users	Получение списка пользователей с фильтром
PUT	/api/users	Редактирование данных пользователя

Разработанный API включает несколько функциональных групп методов. Подсистема авторизации обеспечивает регистрацию пользователей и выполнение входа в систему. Административный интерфейс предоставляет возможность управления учетными записями пользователей и изменения их ролей. Для работы с данными производства реализованы методы получения информации о партиях металла, технологических параметрах оборудования, результатах лабораторных исследований и агрегированной статистике. Отдельные интерфейсы предназначены для пользователей по ролям, что позволяет предоставлять каждому типу пользователей только необходимый набор данных. Для работы с системой уведомлений реализованы методы получения активных уведомлений и изменения их значений. Дополнительно предусмотрен интерфейс формирования отчетов по партиям металла.

3.3.3 Websocket

Для оперативного отображения изменений в пользовательском интерфейсе в системе реализован механизм двустороннего обмена данными на основе технологии WebSocket.

После открытия клиентского приложения устанавливается постоянное соединение между браузером пользователя и сервером через специализированную конечную точку WebSocket. Для организации обмена сообщениями используется протокол STOMP, обеспечивающий удобную маршрутизацию данных по тематическим каналам.

В системе реализовано четыре канала передачи данных: для распространения уведомлений о возникновении отклонений у параметров новых записей; для передачи изменений, связанных с данными MES; для обновления результатов лабораторных исследований; для передачи новых технологических параметров оборудования. После получения новых данных из Kafka и их сохранения в базе данных сервер формирует соответствующее сообщение и публикует его в необходимый WebSocket-канал. Все подключенные клиенты автоматически получают данное сообщение без выполнения дополнительных HTTP-запросов.

На клиентской стороне поступающие события используются для актуализации отображаемой информации. В зависимости от типа сообщения выполняется обновление статистических показателей, списков уведомлений, сведений о производственных партиях, результатов лабораторных исследований и графиков технологических параметров. Для данных SCADA реализовано динамическое добавление новых точек на графики без полной перезагрузки страницы, что обеспечивает плавное отображение технологических процессов в режиме реального времени.

Таким образом, разработанная архитектура обеспечивает безопасное, масштабируемое и отказоустойчивое взаимодействие между компонентами информационной системы.

3.4 Реализация серверной части

Серверная часть информационной системы витрины качества металла реализована в отдельном Maven-проекте, построенном по слоистой архитектуре. Каждый слой (сущности, репозитории, сервисы, контроллеры, конфигурации) размещён в отдельном пакете, что обеспечивает чёткое разграничение ответственности между компонентами и упрощает сопровождение кода в долгосрочной перспективе.

3.4.1 Модульная структура проекта

Архитектура серверного приложения содержит следующие ключевые пакеты:

- пакет `configurations` содержит классы конфигурации приложения: настройку потребителей Apache Kafka, конфигурацию WebSocket с регистрацией конечных точек STOMP-брокера для передачи уведомлений и настройку правил CORS для взаимодействия с клиентским приложением;
- пакет `controllers` содержит REST-контроллеры, реализующие HTTP-интерфейс серверного приложения. Каждый контроллер соответствует конкретному ресурсу или функциональной области системы;
- пакет `customExceptions` содержит пользовательские исключения приложения и централизованный обработчик, настроенный на формирование ответов с необходимыми HTTP-статусами;
- пакет `dto` содержит объекты передачи данных (DTO), используемые для формирования запросов и ответов API. DTO изолируют внутреннюю модель данных от внешнего представления, что позволяет формировать различный набор полей в зависимости от роли пользователя или назначения запроса. Внутри пакета выделены подпакеты, распределяющие классы DTO по областям применения;
- пакет `listeners` содержит классы-потребители (Kafka Consumer) для

- каждого из топиков брокера сообщений;
- пакет `models` содержит JPA-сущности, соответствующие таблицам базы данных. Каждая сущность аннотирована `@Entity` и `@Table` с указанием имени таблицы;
- пакет `repositories` содержит интерфейсы `Spring Data JPA`, расширяющие `JpaRepository` для каждой сущности базы данных;
- пакет `security` содержит компоненты системы безопасности: конфигурацию `Spring Security`, класс JWT-фильтра, реализацию `UserDetailsService` для загрузки данных пользователя из базы данных и вспомогательный сервис для генерации и валидации JWT-токенов;
- пакет `services` содержит сервисные классы, инкапсулирующие бизнес-логику приложения: обработку поступающих из `Kafka` сообщений, формирование ролевых выборок данных, проверку отклонений и генерацию PDF-отчётов.

Таким образом, слоистая организация пакетов серверного приложения обеспечивает строгое разграничение ответственности между уровнями системы и создаёт архитектурную основу для независимого расширения каждого из модулей без изменения смежных слоёв.

3.4.2 Реализация моделей

Модели данных образуют фундамент серверной части информационной системы, поскольку именно через них объекты предметной области отображаются в реляционные таблицы базы данных. В разработанном приложении модели реализованы с использованием технологии JPA (`Java Persistence API`), обеспечивающей объектно-реляционное отображение и позволяющей работать с записями базы данных как с полноценными объектами языка программирования `Java`.

Система включает следующие ключевые модели, покрывающие функциональные требования к хранению данных:

- `DimBatchEntity` (таблица `dim_batch`) – партия металла. Хранит

- строковый идентификатор партии (batchId, первичный ключ), тип металла (MetalTypeEnum, атрибут metalType), временные метки этапов производственного цикла (startTime – начало поступления, processingTime – начало обработки, analysesTime – начало лабораторного анализа, endTime – завершение), текущий статус партии (MesProcessStatusEnum, атрибут processStatus) и показатель выхода годной продукции в процентах (outputYield);
- MesEntity (таблица fact_mes) – производственные параметры обработки партии. Связана с DimBatchEntity отношением @OneToOne по полю batch_id, атрибут DimBatchEntity batch. Содержит идентификаторы записи (recordId, первичный ключ), производственного заказа (orderId), оборудования (equipmentId) и оператора (operatorId), технологические параметры: масса шихты (chargeMass), масса продукта (outputMass) и продолжительность обработки в минутах (durationMin);
 - LimsEntity (таблица fact_lims) – информация о лабораторном анализе. Связана с DimBatchEntity отношением @ManyToOne, поскольку для одной партии может быть проведено несколько анализов (атрибут DimBatchEntity batch). Содержит идентификаторы записи (recordId, первичный ключ), пробы (sampleId), метод анализа (analysisMethod), дату проведения испытаний (testDate) и итоговый статус (LimsStatusEnum: APPROVED, REJECTED);
 - LimsResultEntity (таблица fact_lims_results) – детализированные результаты измерений по каждому показателю анализа. Связана с LimsEntity отношением @ManyToOne с FetchType.LAZY для предотвращения автоматической загрузки всех результатов при запросе LimsEntity. Содержит наименование параметра (parameterName), фактическое значение (value), единицу измерения (unit) и признак соответствия норме (normal);

- ScadaEntity (таблица fact_scada) – измерения датчиков с временными метками. Использует составной первичный ключ из полей recordId и time, описанный отдельным классом ScadaEntityId, реализующим Serializable. Содержит идентификаторы датчика (sensorId) и оборудования (equipmentId), временную метку измерения (time), наименование параметра (parameter), числовое значение (value), единицу измерения (unit) и статус состояния (ScadaStatusEnum: NORMAL, WARNING, ALARM);
- NotificationEntity (таблица fact_notifications) – уведомления об инцидентах, зафиксированных системой при обнаружении отклонений. Содержит текст уведомления (message), идентификаторы оборудования (equipmentId) и датчика (sensorId), источник сигнала (signalSource), уровень критичности (NotificationSeverityEnum: INFO, WARNING, ALARM), статус обработки (NotificationStatusEnum: CREATED, IN_PROGRESS, FALSE_POSITIVE, RESOLVED), флаг просмотра (viewed), комментарий пользователя (comment), временные метки создания (createdAt) и последнего обновления (updatedAt), а также логин пользователя, изменившего статус (updatedBy);
- UserEntity (таблица users) – учётные данные пользователей системы. Использует UUID в качестве первичного ключа. Содержит логин (username), хеш пароля (password), имя (name), фамилию (surname), отчество (patronymic), электронную почту (email), роль (UserRoleEnum: LABORATORY, PRODUCTION, MANAGEMENT, ADMIN) и дату создания учётной записи (createdAt).

Применение технологии JPA позволяет работать с объектами предметной области напрямую, делегируя Hibernate задачи генерации SQL-запросов и управления транзакциями. А реализованные модели данных обеспечивают полное представление основных объектов предметной области

и их взаимосвязей.

3.4.3 Реализация сервисов

Выделение отдельного сервисного слоя позволяет полностью отделить бизнес-логику приложения от механизмов хранения данных и обработки HTTP-запросов. Каждый сервис инкапсулирует набор операций, относящихся к конкретной предметной области, что обеспечивает повторное использование кода, упрощает тестирование и создаёт условия для независимого развития отдельных функциональных модулей системы.

`AdminService` отвечает за операции управления пользователями в административном разделе системы. Предоставляет методы для получения постраничного списка всех учётных записей и для редактирования выбранной записи: смены роли и обновления персональных данных.

`AuthService` обрабатывает запросы аутентификации и регистрации. При аутентификации проверяет переданные логин и пароль через `AuthenticationManager`, при успешной проверке генерирует JWT-токен с идентификатором пользователя, его ролью и временем истечения и возвращает токен клиентскому приложению. При неуспешной аутентификации выбрасывает исключение, транслируемое обработчиком в HTTP-статус 401. Метод регистрации создаёт новую запись в базе данных с предварительным хешированием пароля.

`BatchService` обеспечивает получение информации о партиях металла. Включает методы, доступные всем авторизованным пользователям, а также специализированные методы для определённых ролей системы. Отвечает за получение: списка всех типов металлов со сводной статистикой по количеству партий в каждом статусе; статистики по браку в разрезе типов металлов; списка последних зарегистрированных партий; постраничного списка партий по заданному типу металла с поддержкой фильтрации по статусу и поиска по номеру партии; полной детальной информации по отдельной партии. Помимо этого, содержит метод сохранения новой записи партии в базу данных.

LimsService предоставляет методы для получения выборки последних анализов, списка всех испытаний по конкретной партии как с детализированными результатами, так и без них, а также методы для создания новых объектов LimsEntity и LimsResultEntity.

MesService отвечает за работу с данными производственных операций. Предоставляет методы получения производственных параметров для конкретной партии металла, а также методы сохранения и выдачи сущностей из базы данных.

NotificationService обеспечивает управление уведомлениями об отклонениях. Предоставляет методы для выдачи выборки уведомлений, ещё не принятых в работу; получения списка инцидентов с фильтрацией по статусу; создания нового инцидента; обновления статуса существующего инцидента и маркировки уведомлений как прочитанных. Кроме того, сервис реализует аналитические методы: подсчёт количества уведомлений по конкретной партии и распределение инцидентов по статусам за текущий день.

ReportService реализует серверную генерацию PDF-отчёта по производственной партии. Принимает идентификатор партии, параллельно запрашивает из базы данных все необходимые данные (dim_batch, fact_mes, fact_scada, fact_lims, fact_lims_results, fact_notifications), подсчитывает количество предупреждений и тревог по каждому источнику данных и формирует PDF-документ с помощью библиотеки iText 5. Сформированный документ содержит титульный блок с реквизитами партии, данные об авторе документа и дате его создания, таблицы технологических параметров MES, агрегированные показатели SCADA с указанием количества отклонений, детализацию результатов LIMS по каждому параметру и итоговое заключение о качестве партии.

ScadaService предоставляет методы выборки данных SCADA для отображения на детальной странице партии. Для партий в статусе PROCESSING возвращает актуальные показания датчиков за последнюю

минуту по идентификатору оборудования с сохранением сведений об отклонениях. Для партий, завершивших обработку, выполняет агрегирующий запрос, разбивающий весь период обработки (от `processingTime` до `endTime`) на 60 временных интервалов: для каждого интервала вычисляются средние значения параметров, а данные со статусами `WARNING` и `ALARM` выделяются в отдельные блоки. Для пользователей роли «Производство» каждый параметр формирует собственный набор данных, используемый для построения графиков на детальной странице. Для остальных ролей вычисляются агрегированные показатели – среднее, минимальное, максимальное значения и общее количество превышений нормы. Помимо этого, сервис управляет созданием и выдачей записей из базы данных.

`UserProfileService` отвечает за получение данных профиля текущего пользователя на основании идентификатора, извлечённого из JWT-токена входящего запроса.

`WebSocketService` содержит методы для отправки специализированных сообщений по выделенным WebSocket-каналам, обеспечивая адресную доставку уведомлений подключённым клиентам.

Реализованный сервисный слой обеспечивает единую точку реализации бизнес-логики системы: обработку поступающих производственных данных, выполнение аналитических расчётов, формирование отчётности и управление жизненным циклом уведомлений. Изоляция бизнес-логики от смежных слоёв исключает её дублирование в контроллерах и репозиториях, а однозначная ответственность каждого сервиса обеспечивает соответствие архитектуры принципу единственной ответственности (Single Responsibility Principle).

3.4.4 Реализация контроллеров

Для реализации HTTP-интерфейса серверной части используются REST-контроллеры, построенные на основе модуля `Spring Web`. Контроллеры, аннотированные `@RestController`, обеспечивают обработку входящих HTTP-запросов и автоматическое преобразование объектов приложения в формат

JSON посредством библиотеки Jackson.

AdminController (маршрут /api/admin) реализует административные функции управления пользователями системы. Контроллер предоставляет инструменты поиска и фильтрации учётных записей, а также редактирования персональных данных и изменения ролей пользователей.

AuthController (маршрут /api/auth) отвечает за процессы аутентификации и регистрации. Контроллер обеспечивает проверку учётных данных, выдачу JWT-токенов доступа и создание новых учётных записей. Маршруты данного контроллера являются публично доступными и не требуют предварительной аутентификации.

CommonController (маршрут /api) объединяет общесистемные маршруты, доступные авторизованным пользователям независимо от их роли. Контроллер выступает единой точкой доступа к информации, необходимой различным категориям пользователей при работе с витриной качества металла. Функциональность контроллера охватывает получение сводной информации по типам металлов и производственным партиям, выдачу детальных сведений о выбранной партии, получение данных MES, результатов лабораторных исследований LIMS и агрегированных показателей SCADA. Кроме того, через данный контроллер осуществляется получение данных профиля текущего пользователя и формирование PDF-отчёта по выбранной производственной партии.

LaboratoryController (маршрут /api/laboratory) отвечает за предоставление специализированного функционала сотрудникам лаборатории. Контроллер обеспечивает получение списка последних выполненных анализов, а также детальной информации о результатах лабораторных исследований для конкретной производственной партии, включая показатели контроля качества продукции.

ManagementController (маршрут /api/management) реализует функциональность, предназначенную для руководящего состава предприятия.

Основной задачей контроллера является предоставление агрегированной аналитической информации по производственным показателям и качеству выпускаемой продукции. Через данный интерфейс формируются статистические данные, содержащие сведения о количестве партий в разрезе типов металла, распределении партий по статусам и показателях брака.

ProductionController (маршрут /api/production) содержит маршруты, предназначенные для пользователей производственных подразделений. Контроллер обеспечивает получение сведений о последних производственных партиях, находящихся в обработке на предприятии, а также предоставление детализированных данных SCADA для построения графиков технологических параметров оборудования.

NotificationController (маршрут /api/notifications) управляет жизненным циклом уведомлений об обнаруженных отклонениях и инцидентах. Помимо получения списков уведомлений и статистических данных, контроллер обеспечивает изменение статуса инцидентов, фиксацию ответственных сотрудников и сохранение комментариев по результатам анализа причин возникших отклонений.

Разграничение контроллеров по бизнес-направлениям предприятия обеспечивает логическую изоляцию HTTP-маршрутов, упрощает контроль доступа на уровне Spring Security и создаёт структурную основу для дальнейшего расширения функциональности системы без изменения существующих интерфейсов.

3.4.5 Реализация системы безопасности

Витрина качества металлов обеспечивает доступ к производственным данным различных подразделений предприятия, поэтому одним из ключевых требований к разрабатываемой системе является обеспечение информационной безопасности. Для решения данной задачи реализована централизованная подсистема безопасности на основе фреймворка Spring Security, реализующая stateless-аутентификацию с использованием JWT-

токенов. Отсутствие серверных сессий позволяет обслуживать горизонтально масштабируемые развёртывания.

Конфигурационный класс `SecurityConfig` аннотирован `@Configuration` и `@EnableWebSecurity` и определяет цепочку фильтров безопасности (`SecurityFilterChain`). В цепочке настроено отключение CSRF-защиты (избыточной при использовании JWT), разрешение CORS для клиентского приложения, политика управления сессиями `SessionCreationPolicy.STATELESS` (запрет создания HTTP-сессий) и правила доступа к маршрутам: публичный доступ к `/api/auth/`, доступ к `/api/users/` только для пользователей с ролью `ADMIN`, доступ ко всем остальным маршрутам только для аутентифицированных пользователей. В цепочку фильтров добавлен `JwtAuthenticationFilter`, располагающийся перед `UsernamePasswordAuthenticationFilter`.

Класс `JwtAuthenticationFilter` расширяет `OncePerRequestFilter` и выполняется один раз при каждом HTTP-запросе. При обработке запроса фильтр извлекает значение заголовка `Authorization`, проверяет наличие и формат `Bearer`-токена, передаёт токен в `JwtService` для валидации. При успешной валидации извлекает из `claims` токена идентификатор пользователя и его роль, загружает объект `UserDetails` через `UserDetailsService`, создаёт объект `UsernamePasswordAuthenticationToken` и сохраняет его в `SecurityContextHolder`. Это позволяет нижестоящим компонентам (сервисам и контроллерам) получить данные авторизованного пользователя через `SecurityContextHolder.getContext().getAuthentication()`.

Класс `JwtService` отвечает за генерацию и валидацию JWT-токенов с использованием библиотеки `JWT`. При генерации токена в `claims` записываются идентификатор пользователя (`sub`), его роль, время выдачи (`iat`) и время истечения (`exp`). Метод валидации проверяет целостность подписи, соответствие алгоритма и срок действия токена. Секретный ключ подписи хранится в конфигурационном файле `application.properties` и передаётся в

JwtService через @Value-аннотацию.

Класс CustomUserDetailsService реализует интерфейс UserDetailsService Spring Security и переопределяет метод loadUserByUsername. При вызове метода он извлекает из базы данных объект UserEntity по логину через UserRepository, преобразует его в объект User (реализацию UserDetails) с установкой роли в виде GrantedAuthority и признака активности учётной записи. Если пользователь не найден, выбрасывается UsernameNotFoundException.

Реализованная подсистема безопасности на основе Spring Security и JWT обеспечивает надёжную защиту производственных данных без сохранения состояния на сервере. Хранение роли пользователя непосредственно в подписанном токене исключает возможность подмены прав доступа на клиентской стороне и позволяет серверному приложению выполнять авторизацию при каждом запросе без дополнительных обращений к базе данных.

3.5 Реализация клиентской части

Клиентская часть информационной системы реализована в виде одностраничного приложения (SPA) на базе библиотеки React с использованием Vite в качестве инструмента сборки. Архитектура клиентской части обеспечивает ролевой доступ к производственным данным для четырёх категорий пользователей.

3.5.1 Модульная структура проекта

В React-приложениях распространённым подходом является разделение исходного кода по функциональным зонам ответственности. Такой подход позволяет изолировать пользовательские интерфейсы, механизмы взаимодействия с сервером, логику маршрутизации и переиспользуемые компоненты. Исходный код клиентской части системы расположен в директории src и организован в следующие директории.

Директория pages содержит компоненты верхнего уровня,

соответствующие маршрутам приложения: страницы входа и регистрации, ролевые главные страницы, страница партий по типу металла, детальная страница партии, административный раздел, страница профиля пользователя, страница реестра уведомлений об отклонениях и специализированная страница для отображения ошибок навигации.

Директория `components` содержит переиспользуемые React-компоненты, организованные по принадлежности к конкретным страницам либо в поддиректорию `general` для общесистемных элементов.

Директория `api` содержит функции для взаимодействия с серверным REST API.

Директория `websocket` содержит скрипт управления WebSocket-подпиской на уведомления, а также компонент `WebSocketProvider`, в котором реализована логика интеграции входящих сообщений с состоянием кэша TanStack React Query.

Файл `App.jsx` является корневым компонентом приложения. Он оборачивает всё дерево компонентов в провайдеры `QueryClientProvider`, `WebSocketProvider` и `BrowserRouter`, содержащий определения обычных маршрутов `Route` и защищённых маршрутов `ProtectedRoute`.

Модульная организация клиентского приложения обеспечивает низкую связанность компонентов, единственную ответственность каждого модуля и удобную навигацию по структуре проекта. Разделение слоя взаимодействия с API от слоя представления позволяет изменять логику запросов без затрагивания интерфейсных компонентов.

3.5.2 Реализация общих компонентов

`ProtectedRoute` — компонент-обёртка для маршрутов, требующих аутентификации и определённой роли. При рендеринге проверяет наличие JWT-токена и соответствие роли пользователя списку разрешённых ролей для данного маршрута. При несоответствии выполняется перенаправление на страницу входа или на страницу с сообщением об отсутствии доступа.

Footer – компонент, встраивающийся в нижнюю часть каждой страницы после авторизации. Содержит краткую информацию о системе и кнопку прокрутки страницы в начало.

Header – компонент верхней навигационной панели, присутствующий на всех страницах после входа в систему. Отображает наименование системы, навигационные ссылки на разделы, доступные пользователю в соответствии с его ролью (компонент HeaderLinkButton), иконку уведомлений с числовым счётчиком непрочитанных сообщений, обновляемым в реальном времени через WebSocket (для пользователей ролей «Производство» и «Руководство»), кнопку перехода в профиль и кнопку выхода из системы.

HeaderIconButton – компонент кнопки-иконки, принимающий обработчик события, иконку и поддерживающий состояние для отображения бейджа-индикатора, привлекающего внимание пользователя.

NotificationDropdown – выпадающая панель, содержащая список последних инцидентов (компонент NotificationItem), ещё не принятых в работу, и ссылку на полный реестр уведомлений. Компонент навигационной панели и выпадающая панель уведомлений представлены на рисунке 9.

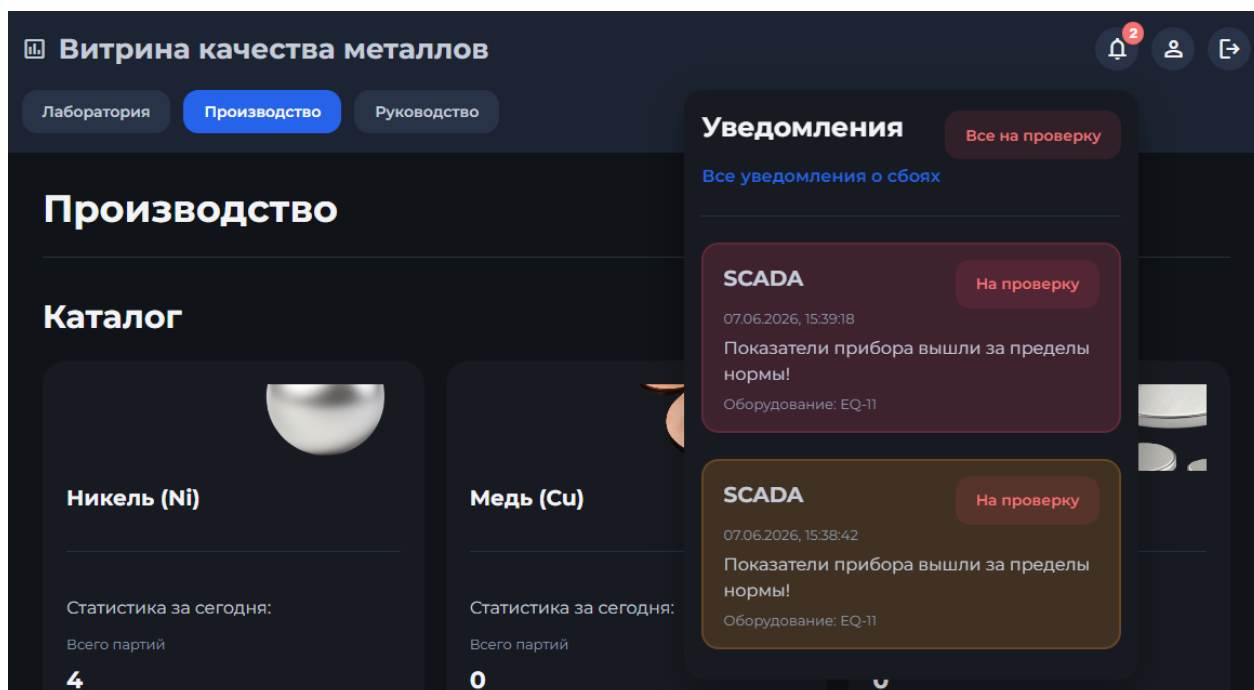


Рисунок 9 – Навигационная панель системы и выпадающая панель уведомлений

PaginationButton – компонент постраничной навигации, используемый в табличных представлениях с большим количеством записей. Принимает внутренний компонент, признак активности и обработчик смены страницы.

Loader – компонент-индикатор загрузки, отображаемый в период выполнения запросов к API. Применяется совместно с состоянием isLoading, возвращаемым хуками TanStack React Query.

Реализация общих компонентов позволяет сосредоточить повторно используемую логику и разметку в единственном месте. Любое изменение вносится один раз и автоматически распространяется на все страницы, использующие соответствующий компонент.

3.5.3 Реализация раздела «Главная»

В основе интерфейсной концепции лежит ролевая модель представления информации. Для каждой категории пользователей разработана отдельная витрина данных, содержащая наиболее релевантные показатели и элементы управления.

Главная страница системы (рис. 10) для ролей «Лаборатория», «Производство» и «Руководство» включает общую часть – обзорную статистику по типам металлов и статусам производственных партий.

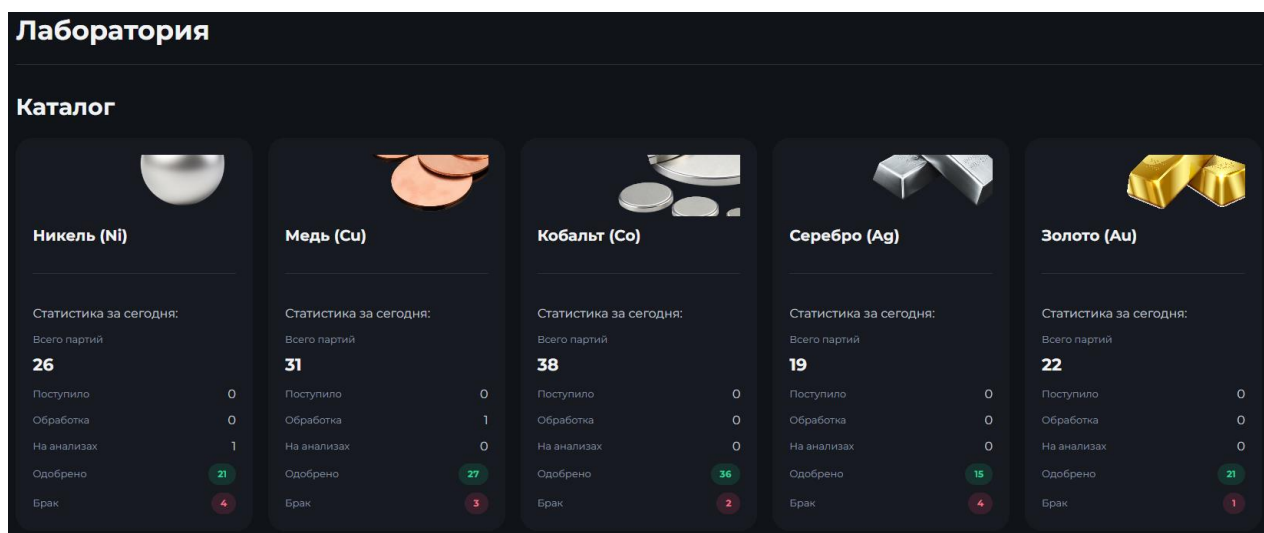


Рисунок 10 – Общая часть главной страница системы

LaboratoryDashboard – витрина для роли «Лаборатория». Страница

состоит из двух зон. Верхняя зона содержит компонент MetalGrid – сетку карточек типов металлов, каждая из которых отображает наименование металла, общее количество партий и мини-статистику по статусам за текущий день в виде счётчиков. При нажатии на наименование выполняется переход на страницу партий соответствующего типа металла. Нижняя зона содержит компонент LastLaboratoryRecordTable (рис. 11) – таблицу последних лабораторных анализов по всем партиям с данными о пробе, типе металла, методе, дате анализа и статусе. Таблица поддерживает переход к детальной странице партии по нажатию кнопки «Подробнее».

Последние анализы					
Проба	Тип металла	Метод	Дата	Статус	
BATCH-f17ca762-a5fa-4f3f-bf65-68f08b292197-SAMPLE-2	Серебро (Ag)	Испытание на ударную вязкость	07.06.2026, 15:24:34	ОДОБРЕНО	Подробнее
BATCH-f17ca762-a5fa-4f3f-bf65-68f08b292197-SAMPLE-1	Серебро (Ag)	Испытание на растяжение	07.06.2026, 15:24:34	ОДОБРЕНО	Подробнее
BATCH-8826f66c-d87a-4342-b6b7-28ed1691f188-SAMPLE-0	Никель (Ni)	Испытание на ударную вязкость	07.06.2026, 15:22:17	ОДОБРЕНО	Подробнее
BATCH-6abddb19-e2bd-4931-bd80-dfbd3e940e5a-SAMPLE-1	Родий (Rh)	Рентгенофлуоресцентный анализ	07.06.2026, 15:21:12	ОДОБРЕНО	Подробнее

Рисунок 11 – Компонент LastLaboratoryRecordTable

ProductionDashboard – витрина для роли «Производство». Верхняя зона аналогично содержит MetalGrid. Нижняя зона содержит компонент LastProductionRecordTable (рис. 12) – таблицу последних партий с отображением номера партии, типа металла, идентификатора оборудования, дат поступления и завершения анализов, статуса и кнопки перехода на детальную страницу.

Последние партии					
Партия	Тип металла	ID оборудования	Поступление	Окончание анализов	Статус
BATCH-847d0a5b-c70f-4fe5-aba8-16881c4da90e	Серебро (Ag)	EQ-1	07.06.2026, 15:24:34		ОБРАБОТКА Подробнее
BATCH-97d6307b-cf2f-4a81-ba63-a92957cbd1e4	Иридий (Ir)	EQ-14	07.06.2026, 15:22:43		ОБРАБОТКА Подробнее
BATCH-f17ca762-a5fa-4f3f-bf65-68f08b292197	Серебро (Ag)	EQ-13	07.06.2026, 15:18:24	07.06.2026, 15:24:34	ОДОБРЕНО Подробнее
BATCH-8826f66c-d87a-4342-b6b7-28ed1691f188	Никель (Ni)	EQ-19	07.06.2026, 15:18:24	07.06.2026, 15:22:17	ОДОБРЕНО Подробнее

Рисунок 12 – Компонент LastProductionRecordTable

ManagementDashboard – витрина для роли «Руководство». Помимо MetalGrid содержит компонент LastMetalStatisticsGrid (рис. 13) – аналитический блок с агрегированными показателями: общей долей брака от всех завершённых партий и средним выходом годного по типам металлов.

Статистика по металлам			
Серебро (Ag)	Иридий (Ir)	Никель (Ni)	Родий (Rh)
Данные за последнюю неделю: Обработано партий 2 Средний выход годного 98.43% Процент брака 0.00%	Данные за последнюю неделю: Обработано партий 1 Средний выход годного 98.32% Процент брака 0.00%	Данные за последнюю неделю: Обработано партий 2 Средний выход годного 99.00% Процент брака 0.00%	Данные за последнюю неделю: Обработано партий 1 Средний выход годного 99.37% Процент брака 0.00%

Рисунок 13 – Компонент LastMetalStatisticsGrid

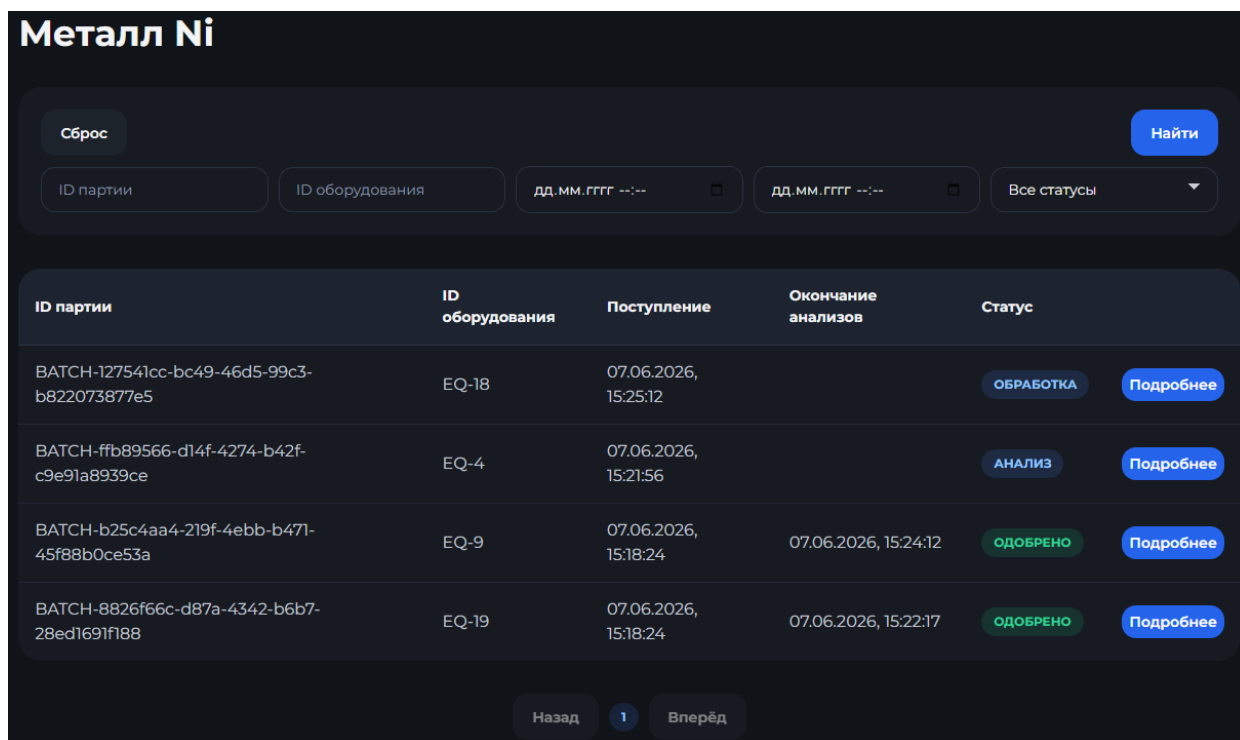
Компоненты MetalGrid, LastLaboratoryRecordTable, LastProductionRecordTable и LastMetalStatisticsGrid получают данные через хуки useQuery библиотеки TanStack React Query. Механизм автоматического фонового обновления кэша позволяет пользователю видеть актуальные производственные данные без ручного обновления страницы.

Ролевая архитектура витрин позволяет каждому подразделению

работать с информацией в том представлении, которое соответствует его производственным задачам.

3.5.4 Реализация раздела «Партии металла»

Раздел «Партии металла» реализован в компоненте MetalPage, открываемом по маршруту /batches/:metalType. Страница выполняет запрос к /api/batches/:metalType и отображает полный список партий выбранного типа металла, что представлено на рисунке 14.



The screenshot shows a web interface for 'Металл Ni'. At the top, there is a search bar with a 'Сброс' (Reset) button on the left and a 'Найти' (Find) button on the right. Below the search bar are four input fields: 'ID партии' (Batch ID), 'ID оборудования' (Equipment ID), two date pickers labeled 'ДД.ММ.ГГГГ --:--', and a dropdown menu labeled 'Все статусы' (All statuses). The main part of the interface is a table with the following columns: 'ID партии', 'ID оборудования', 'Поступление' (Arrival), 'Окончание анализов' (End of analysis), and 'Статус' (Status). The table contains four rows of data. Each row has a 'Подробнее' (More details) button next to the status. At the bottom of the table, there is a pagination bar with 'Назад' (Back), '1', and 'Вперёд' (Forward) buttons.

ID партии	ID оборудования	Поступление	Окончание анализов	Статус
BATCH-127541cc-bc49-46d5-99c3-b822073877e5	EQ-18	07.06.2026, 15:25:12		ОБРАБОТКА
BATCH-ffb89566-d14f-4274-b42f-c9e91a8939ce	EQ-4	07.06.2026, 15:21:56		АНАЛИЗ
BATCH-b25c4aa4-219f-4ebb-b471-45f88b0ce53a	EQ-9	07.06.2026, 15:18:24	07.06.2026, 15:24:12	ОДОБРЕНО
BATCH-8826f66c-d87a-4342-b6b7-28ed1691f188	EQ-19	07.06.2026, 15:18:24	07.06.2026, 15:22:17	ОДОБРЕНО

Рисунок 14 – Страница типа металла

Под заголовком расположен блок фильтрации MetalSearchPanel. Параметры активного фильтра включаются в тело запроса, что вызывает автоматическую повторную выборку данных.

Основным элементом страницы является компонент MetalBatchesTable – таблица партий со столбцами: номер партии, идентификатор оборудования, дата поступления, дата завершения и текущий статус. Каждая строка содержит кнопку перехода на детальную страницу партии. В нижней части таблицы расположен компонент PaginationButton для постраничной навигации.

Концентрация всей логики фильтрации и постраничного отображения на

одной странице позволяет пользователю эффективно находить нужную партию в условиях большого объёма производственных данных.

3.5.5 Реализация раздела «Детальная информация партии»

Раздел «Детальная информация партии» реализован в компоненте BatchPage, доступном по маршруту `/:role/batches/:batchId`. Он содержит консолидированную информацию из MES, SCADA и LIMS. Пользователь получает доступ к технологическим параметрам обработки и результатам лабораторных исследований в детализированном или агрегированном виде в зависимости от роли.

Общие данные для всех пользователей содержатся в компоненте BatchDataPanel (рис. 15): хронологический таймлайн смены статусов партии с временными метками и длительностью каждого этапа, итоговый статус продукции и технологические параметры из MES.

Партия #BATCH-5a4d9764-27d9-4f37-9b9e-1956924ffb35 СОЗДАТЬ ОТЧЁТ

Общая информация

ID партии	Тип металла	Поступление	Обработка	Начало анализ	Окончание анализ	Статус
BATCH-5a4d9764-27d9-4f37-9b9e-1956924ffb35	Кобальт (Co)	09.06.2026, 15:39:46	09.06.2026, 15:40:00	09.06.2026, 15:43:56	09.06.2026, 15:45:22	ОДОБРЕНО

Производственные данные MES

ID произв. заказа	ID оборудования	ID оператора	Масса шихты, т	Масса продукта, т	Выход годного, %	Время обработки, мин
ORDER-71c98387-b7a0-4df3-a40e-ec05ff084a7d	EQ-9	OP-173	40.59	32.88	81.01	612

Рисунок 15 – Детальная страница партии

В зависимости от роли авторизованного пользователя в BatchPage встраивается один из ролевых компонентов — BatchLaboratoryPanel, BatchProductionPanel или BatchManagementPanel.

Для роли «Лаборатория» реализована полная детализация лабораторных анализов (рис. 16): для каждого анализа отображается метод, дата, статус и таблица всех измеренных параметров с фактическими значениями, единицами измерения и цветовым выделением отклонений от нормы.

Лабораторные анализы

LIMS

Анализ #1

ОДОБРЕНО

ID пробы

BATCH-b25c4aa4-219f-4ebb-b471-45f88b0ce53a-SAMPLE-1

Метод

Металлографический анализ

Дата анализа

07.06.2026, 15:24:12

Параметры анализа

Параметр	Значение	Ед. изм.	Соотв. норме
Наличие дефектов	нет		✓
Однородность	высокая		✓
Размер зерна	31.084	мкм	✓

Рисунок 16 – BatchLaboratoryPanel – блок анализов

Для ролей «Производство» и «Руководство» отображается только заголовочная информация анализов: метод, дата и итоговый статус (рис. 17).

Лабораторные анализы

LIMS

ID пробы	Метод	Дата анализа	Статус
BATCH-f17ca762-a5fa-4f3f-bf65-68f08b292197-SAMPLE-2	Испытание на ударную вязкость	07.06.2026, 15:24:34	ОДОБРЕНО
BATCH-f17ca762-a5fa-4f3f-bf65-68f08b292197-SAMPLE-1	Испытание на растяжение	07.06.2026, 15:24:34	ОДОБРЕНО
BATCH-f17ca762-a5fa-4f3f-bf65-68f08b292197-SAMPLE-0	Рентгенофлуоресцентный анализ	07.06.2026, 15:24:34	ОДОБРЕНО

Рисунок 17 – BatchProductionPanel, BatchManagementPanel – блок анализов

Для роли «Производство» реализована полная детализация показателей

оборудования с помощью графиков Recharts (рис. 18). В период обработки партии новые данные SCADA поступают в компонент через WebSocket и встраиваются в нужный график посредством метода `queryClient.setQueryData`, что исключает необходимость повторных HTTP-запросов. Дополнительно реализована функция `optimizeRealtimePoints`, которая удаляет из буфера точки, зафиксированные более минуты назад, сохраняя при этом все значения, отмеченные как отклонения.

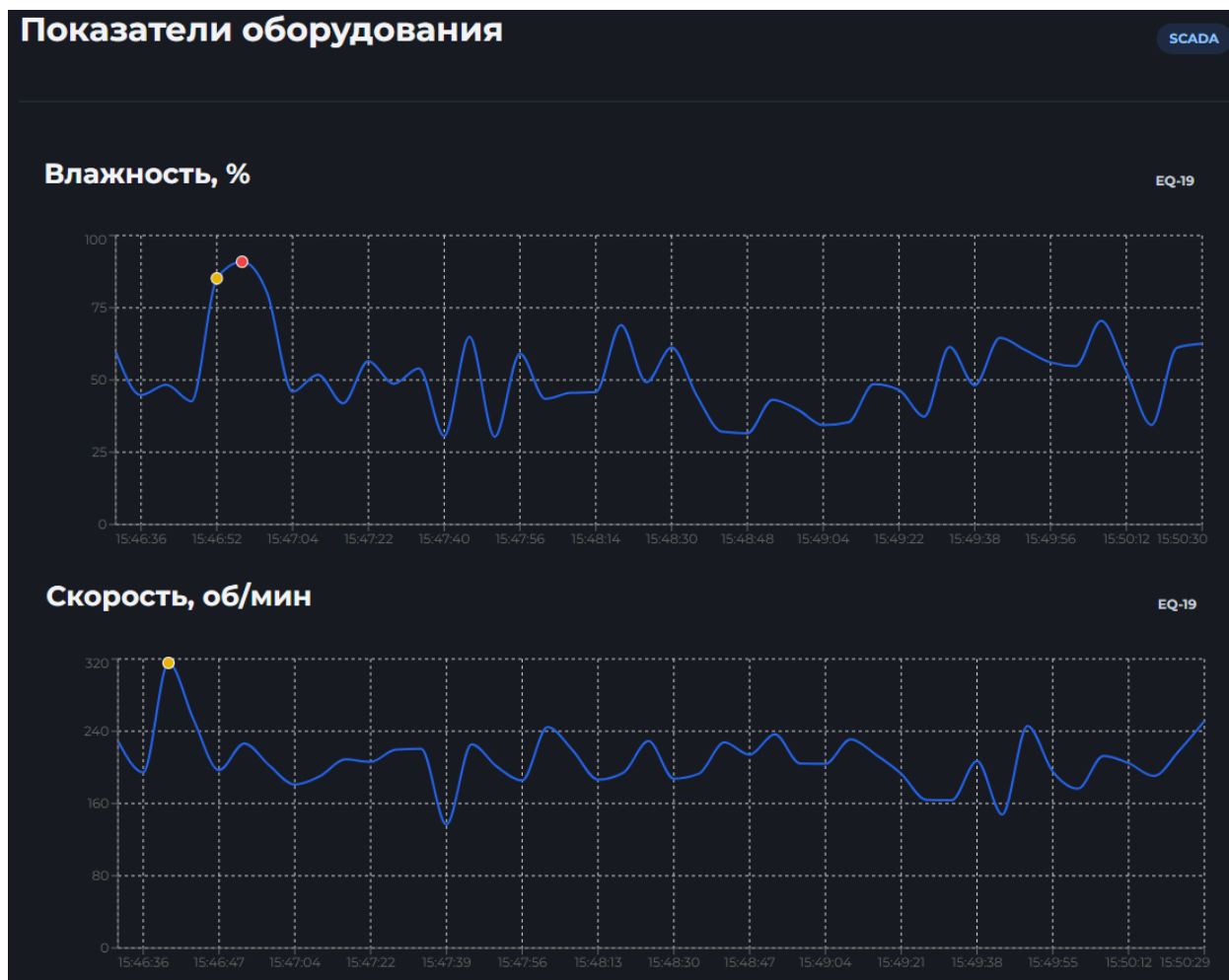


Рисунок 18 – BatchProductionPanel – блок показателей оборудования

Для ролей «Лаборатория» и «Руководство» отображается агрегированная информация о показателях SCADA и общее количество зафиксированных отклонений от нормы для партии, что можно рассмотреть на рисунке 19.

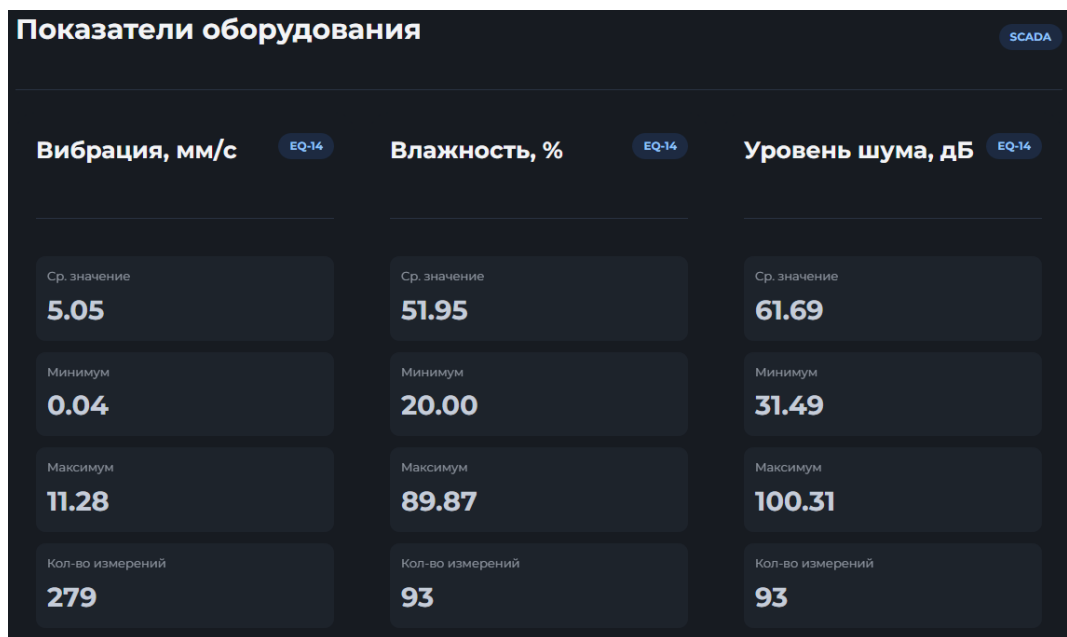


Рисунок 19 – BatchLaboratoryPanel, BatchManagementPanel – блок показателей оборудования

На детальной странице партии также предусмотрена функция автоматической генерации PDF-отчёта. После нажатия кнопки «Создать отчёт» и получения ответа от сервера пользователь автоматически переходит на страницу сформированного PDF-файла (рис. 20), содержащего полные сведения для проведения производственных аудитов.

ОТЧЁТ ПО ПАРТИИ	
BATCH-4009f622-0d32-4a31-83cd-f2778c0ccbfb	
ИНФОРМАЦИЯ ОБ ОТЧЁТЕ	
Автор:	Жерновников Никита Олегович
Дата создания:	09.06.2026 16:00
ОТКЛОНЕНИЯ И АВАРИИ	
Аварийные уведомления:	2
Отклонения от норм (LIMS):	2
ИНФОРМАЦИЯ О ПАРТИИ	
Тип металла:	Серебро (Ag)
Поступление:	09.06.2026 15:46
Обработка:	09.06.2026 15:46
Начало анализа:	09.06.2026 15:50
Окончание анализа:	09.06.2026 15:52
Статус:	БРАК
Выход годного:	85.834%
ПРОИЗВОДСТВЕННЫЕ ДАННЫЕ (MES)	
Производственный заказ:	ORDER-f9b16dc-dbb6-450a-b27e-751070c25b91
Оборудование:	EQ-19
Оператор:	OP-217
Масса шихты:	95.637 т
Масса продукта:	82.089 т
Время обработки:	199 мин
ЛАБОРАТОРНЫЕ АНАЛИЗЫ (LIMS)	

Анализ 1			
Проба:	BATCH-4009f622-0d32-4a31-83cd-f2778c0ccbfb-SAMPLE-0		
Метод:	Испытание на твёрдость		
Дата:	09.06.2026 15:52		
Статус:	ОТКЛОНЕНО		
Параметр	Значение	Ед. изм.	Норма
Твёрдость по Бринеллю	SI.644	HB	Нет
Твёрдость по Виккерсу	SI.339	HV	Нет

ПОКАЗАТЕЛИ ОБОРУДОВАНИЯ (SCADA)					
Оборудование	Параметр	Среднее	Мин.	Макс.	Измерений
EQ-19	Вибрация	5.35	0.07	10.84	81
EQ-19	Влажность	50.95	21.04	91.92	121
EQ-19	Скорость	206.8	100.83	327.31	242

Документ сгенерирован автоматически - 09.06.2026 16:00:31

Рисунок 20 – Отчет о партии металла

Детальная страница партии является центральным информационным объектом системы. Предоставленные в ней данные позволяют пользователю получить исчерпывающее представление о состоянии партии без перехода между несколькими системами.

3.5.6 Реализация раздела «Уведомления о сбоях»

Одной из важнейших задач системы является своевременное информирование сотрудников о возникающих отклонениях. Для её решения реализован специализированный раздел – компонент NotificationsPage, предназначенный для централизованного просмотра и обработки уведомлений об инцидентах. Раздел доступен пользователям ролей «Производство» и «Руководство» и представляет собой единый реестр уведомлений, формируемых на основании данных MES и SCADA. Записи создаются автоматически при обнаружении отклонений технологических параметров от нормативных значений. Компонент сетки уведомлений отображен на рисунке 21.

The image displays a dark-themed user interface for a notifications grid. It contains three vertical cards, each representing a different notification. Each card has a header with an icon (exclamation mark in a circle), a number, and a system name. Below the header is a message box, followed by fields for equipment ID, sensor ID, and a status dropdown menu. A comment field is at the bottom of the main content area. At the very bottom of each card is a 'Сохранить' (Save) button. Metadata at the bottom of each card includes creation and update timestamps, and the user who made the change.

Notification #	System	Status	Message	Equipment ID	Sensor ID	Created At	Updated At	Changed By
#6	SCADA	СОЗДАНО	Показатели прибора вышли за пределы нормы!	EQ-11	S-31	07.06.2026, 15:39:18	—	—
#5	SCADA	СОЗДАНО	Показатели прибора вышли за пределы нормы!	EQ-11	S-32	07.06.2026, 15:38:42	—	—
#4	MES	НА ПРОВЕРКЕ	Показатели при обработке партии вышли за пределы нормы!	EQ-11	—	07.06.2026, 15:38:33	07.06.2026, 15:40:31	manager

Рисунок 21 – Компонент NotificationsGrid

Основным элементом страницы является сетка карточек уведомлений – компоненты NotificationsGrid и NotificationsCard, содержащие сведения об источнике события, уровне критичности, статусе обработки, времени возникновения и кратком описании выявленной проблемы. Для повышения удобства восприятия применяется цветовое кодирование уровня опасности инцидента.

Для уточнения выборки реализован компонент фильтрации NotificationsSearchPanel. Пользователь может ограничивать список по источнику данных, статусу обработки, временному интервалу возникновения события и идентификатору оборудования.

В системе реализован полный жизненный цикл обработки уведомлений: после поступления нового события пользователь может принять его в работу, обозначив себя ответственным за анализ ситуации. По результатам проверки инцидент переводится в статус устранённого либо помечается как ложное срабатывание.

Для пользователей роли «Руководство» дополнительно отображается компонент статистики NotificationsStats: количество активных инцидентов и распределение событий по статусам обработки за сегодняшний день.

Реализованный раздел уведомлений обеспечивает полную прослеживаемость каждого инцидента – от момента автоматического обнаружения отклонения до фиксации результатов его устранения.

3.5.7 Реализация раздела «Управление пользователями»

Для реализации административного интерфейса в системе разработан специализированный раздел, доступный исключительно пользователям с ролью администратора. Он реализован в компоненте AdminPage (рис. 22). После открытия страницы выполняется запрос к серверному API для получения списка зарегистрированных пользователей с учётом параметров фильтрации и分页ного отображения данных.

Пользователи

Сброс

Найти

Username

Имя

Фамилия

Все роли

ID: 3e4f0cdf-b490-4425-a995-5cb1ddce1d93

ADMIN

@admin

Фамилия

Парамонова

Имя

Анна

Отчество

Сергеевна

Email

admin@gmail.com

Роль

ADMIN

Создан: 2026-06-07T15:18:21.309526

Сохранить

ID: 11bce815-7620-4c68-97bc-19042d6b110b

MANAGEMENT

@manager

Фамилия

Жерновников

Имя

Никита

Отчество

Олегович

Email

manager@gmail.com

Роль

MANAGEMENT

Создан: 2026-06-07T15:18:21.309526

Сохранить

ID: 0c60d185-c428-4652-ab6e-5a6574774551

LABORATORY

@lab

Фамилия

Волкова

Имя

Ольга

Отчество

Станиславовна

Email

lab@gmail.com

Роль

LABORATORY

Создан: 2026-06-07T15:18:21.309526

Сохранить

Рисунок 22 – Компонент AdminPage

Основным элементом интерфейса является сетка карточек пользователей (компонент AdminUsersGrid), содержащая сведения об учётных записях системы: логин, фамилию, имя, адрес электронной почты, назначенную роль и дату регистрации. В рамках данного компонента администратор может изменять персональные данные пользователя и назначать новую роль в системе.

Для упрощения поиска реализован компонент фильтрации AdminSearchPanel. Администратор может выполнять поиск пользователей по различным атрибутам, что позволяет оперативно находить необходимые учётные записи.

После выполнения операций редактирования библиотека TanStack React Query автоматически инвалидирует связанный кэш и повторно запрашивает

актуальный список пользователей, обеспечивая синхронность отображаемых данных с состоянием базы данных без ручного обновления страницы.

Централизация управления пользователями в отдельном разделе с ролевым ограничением доступа (исключительно для ADMIN) позволяет поддерживать актуальную структуру ролей в системе без прямого взаимодействия с базой данных.

3.6 Итоги реализации системы

Реализованная информационная система витрины качества металла представляет собой полнофункциональный программный комплекс, включающий серверную и клиентскую части, интегрированные посредством REST API и WebSocket-соединения.

Серверная часть обеспечивает обработку входящих производственных данных через брокер сообщений Apache Kafka, хранение данных в реляционной базе данных PostgreSQL, разграничение доступа, обнаружение отклонений технологических параметров, доставку уведомлений через WebSocket, а также генерацию PDF-отчётов по партиям.

Клиентская часть ориентирована на обеспечение удобного и оперативного доступа к производственной информации для различных категорий сотрудников предприятия. Применение ролевых витрин данных, адаптированных к задачам каждого подразделения, в сочетании с WebSocket-уведомлениями в реальном времени и аналитическими визуализациями на основе Recharts позволяет повысить эффективность взаимодействия пользователей с системой.

Весь программный комплекс контейнеризирован с помощью Docker и оркестрирован посредством Docker Compose, что обеспечивает воспроизводимое развёртывание всех сервисов единственной командой и гарантирует идентичность окружений при разработке и эксплуатации.

ЗАКЛЮЧЕНИЕ

В процессе выполнения выпускной квалификационной работы была спроектирована и реализована информационная система витрина качества металлов, предназначенная для консолидации данных из производственных систем предприятия и обеспечения различных категорий пользователей актуальной информацией о ходе технологических процессов и качестве выпускаемой продукции.

Актуальность разработки обусловлена необходимостью повышения оперативности принятия управленческих решений в условиях современного металлургического производства. Данные о технологических процессах формируются одновременно в нескольких информационных системах, что затрудняет их совместный анализ и требует создания единого инструмента для централизованного доступа к производственной информации. Разработанная система решает данную задачу за счёт интеграции данных из систем MES, SCADA и LIMS в рамках единого информационного пространства.

В ходе выполнения работы был проведён анализ предметной области. На основании результатов анализа были сформулированы функциональные и нефункциональные требования к разрабатываемой системе, определены категории пользователей и разработана ролевая модель доступа к данным.

Для реализации серверной части системы был выбран язык программирования Java и фреймворк Spring Boot, позволяющий создавать масштабируемые корпоративные приложения. Архитектура серверного приложения построена на основе многоуровневого подхода и включает слои контроллеров, сервисов, моделей данных и компонентов доступа к базе данных. Для организации безопасного взаимодействия пользователей с системой реализованы механизмы аутентификации и авторизации на основе технологии JWT и средств Spring Security.

В качестве системы управления базами данных была выбрана PostgreSQL. Для хранения производственной информации разработана

структура базы данных, включающая сущности производственных партий, данных MES, SCADA, LIMS, уведомлений об отклонениях и учетных записей пользователей. Для повышения производительности обработки данных реализованы механизмы индексации, партиционирования таблиц и автоматического создания новых секций хранения телеметрической информации.

Особое внимание было уделено вопросам интеграции с внешними производственными системами. Для организации асинхронного обмена сообщениями использован брокер Apache Kafka, обеспечивающий надёжную доставку данных и возможность масштабирования системы при увеличении объёмов производственной информации. Для каждого источника данных были реализованы специализированные каналы обмена сообщениями и механизмы их последующей обработки.

Клиентская часть системы разработана с использованием библиотеки React и построена на основе компонентного подхода. Пользовательский интерфейс реализует ролевую модель представления информации и предоставляет отдельные рабочие пространства для сотрудников лаборатории, производственных подразделений, руководства предприятия и администраторов системы. Для отображения актуальных данных используются современные механизмы управления состоянием приложения и автоматического обновления информации.

Важной особенностью разработанного решения является поддержка работы в режиме реального времени. С использованием технологии WebSocket реализована система оперативной доставки уведомлений и обновлений производственных данных без необходимости постоянного опроса сервера. Это позволяет пользователям своевременно получать информацию о возникновении отклонений технологических параметров и изменениях состояния производственных процессов.

Таким образом, в ходе выполнения выпускной квалификационной работы были достигнуты следующие результаты:

- проведён анализ предметной области и сформулированы требования к информационной системе;
- разработана архитектура программного решения;
- спроектирована и реализована база данных для хранения производственной информации;
- реализованы механизмы индексации и партиционирования, обеспечивающие эффективную обработку больших объёмов данных;
- разработана серверная часть системы;
- реализована интеграция с брокером сообщений Apache Kafka для организации асинхронного обмена данными;
- разработана клиентская часть системы;
- реализованы механизмы обновления данных в режиме реального времени на основе технологии WebSocket;
- разработаны интерфейсы для сотрудников лаборатории, производственных подразделений, руководства предприятия и администраторов системы;
- реализованы средства мониторинга производственных процессов, управления уведомлениями и формирования отчетов.

Разработанная информационная система может быть использована в качестве основы для создания корпоративной платформы мониторинга качества металлопродукции для предприятия Норникель.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ISO 9001:2015. Quality management systems — Requirements [Электронный ресурс]. — Geneva : International Organization for Standardization, 2015. — URL: <https://www.iso.org/standard/62085.html> (дата обращения: 23.04.2026).
2. ISO/IEC 17025:2017. General requirements for the competence of testing and calibration laboratories [Электронный ресурс]. — Geneva : International Organization for Standardization, 2017. — URL: <https://www.iso.org/standard/66912.html> (дата обращения: 23.04.2026).
3. ПАО «ГМК «Норильский никель». Годовой отчёт за 2023 год [Электронный ресурс]. — М., 2024. — URL: <https://ar2023.nornickel.ru> (дата обращения: 23.04.2026).
4. Интегрированные системы проектирования и управления. SCADA : учебное пособие / Х. Н. Музипов, О. Н. Кузяков, С. А. Хохрин [и др.]. — Санкт-Петербург : Лань, 2022. — 408 с. — ISBN 978-5-8114-3265-3. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/213209> (дата обращения: 30.04.2026). — Режим доступа: для авториз. пользователей.
5. Федотов, А. В. Компьютерное управление в производственных системах : Учебное пособие для вузов / А. В. Федотов, В. Г. Хомченко. — 2-е изд., стер. — Санкт-Петербург : Лань, 2021. — 620 с. — ISBN 978-5-8114-8065-4. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/171424> (дата обращения: 30.04.2026). — Режим доступа: для авториз. пользователей.
6. Boyar, K., Pham, A., Swantek, S., Ward, G., Herman, G. Laboratory Information Management Systems (LIMS) // Cannabis Laboratory Fundamentals / ed. by S. R. Opie. — Cham : Springer, 2021. — P. 183–210. — DOI: 10.1007/978-3-030-62716-4_7.

7. AVEVA PI Vision 2025: руководство пользователя [Электронный ресурс] // AVEVA Documentation. — URL: https://docs-be.aveva.com/bundle/pi-vision-2025-user-guide-ru/raw/resource/enus/pi-vision-2025-user-guide-ru.pdf?save_local=false (дата обращения: 02.05.2026).
8. PARCview // dataPARC Knowledge Base [Электронный ресурс]. — URL: <https://kb.dataparc.com/docs/parcview> (дата обращения: 02.05.2026).
9. Primetals Technologies. Quality Management System for Long Products: Product Description. – Linz: Primetals Technologies, 2022. – 62 p.
10. SMS group GmbH. X-Pact Quality – The Integrated Quality Management System. – Düsseldorf: SMS group, 2022. – 28 p.
11. Siemens Opcenter X Quality : [Электронный ресурс]. — Режим доступа: <https://www.siemens.com/ru-ru/products/opcenter/quality-x-cloud-qms/essentials/> (дата обращения: 02.05.2026).
12. Рочев, К. В. Информационные технологии. Анализ и проектирование информационных систем : учебное пособие для вузов / К. В. Рочев. — 4-е изд., стер. — Санкт-Петербург : Лань, 2025. — 128 с. — ISBN 978-5-507-50803-7. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/465164> (дата обращения: 01.05.2026). — Режим доступа: для авториз. пользователей.
13. Остроух, А. В. Проектирование информационных систем : монография / А. В. Остроух, Н. Е. Суркова. — 2-е изд., стер. — Санкт-Петербург : Лань, 2021. — 164 с. — ISBN 978-5-8114-8377-8. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/175513> (дата обращения: 01.05.2026). — Режим доступа: для авториз. пользователей.
14. Копылова, Я. А. Унифицированный язык моделирования UML:

- правила, диаграммы, примеры применения : учебное пособие / Я. А. Копылова, А. Д. Лагунова. — Москва : РТУ МИРЭА, 2025. — 76 с. — ISBN 978-5-7339-2806-7. — Текст : электронный // Лань : электронно-библиотечная система. — URL: <https://e.lanbook.com/book/516288> (дата обращения: 01.05.2026). — Режим доступа: для авториз. пользователей.
15. React Documentation [Электронный ресурс] / Meta Platforms, Inc. — 2026. — URL: <https://react.dev/learn> (дата обращения: 02.05.2026).
 16. React Router Documentation [Электронный ресурс] / Shopify Inc. — 2026. — URL: <https://reactrouter.com/home> (дата обращения: 02.05.2026).
 17. TanStack Query v5 Documentation [Электронный ресурс] / TanStack. — 2026. — URL: <https://tanstack.com/query/v5/docs/framework/react/overview> (дата обращения: 02.05.2026).
 18. Использование Axios с React [Электронный ресурс] // DigitalOcean Community. — URL: <https://www.digitalocean.com/community/tutorials/react-axios-react-ru> (дата обращения: 02.05.2026).
 19. Recharts Documentation [Электронный ресурс] / Recharts Group. — 2026. — URL: <https://recharts.github.io/en-US> (дата обращения: 02.05.2026).
 20. Spring Boot Reference Documentation [Электронный ресурс] / Broadcom Inc. — 2026. — URL: <https://docs.spring.io/spring-boot/reference/index.html> (дата обращения: 02.05.2026).
 21. Spring Security Reference Documentation [Электронный ресурс] / Broadcom Inc. — 2026. — URL: <https://docs.spring.io/spring-security/reference/index.html> (дата обращения: 02.05.2026).
 22. Java JWT: JSON Web Token for Java and Android [Электронный

- ресурс]. — URL: <https://github.com/jwt/jwt> (дата обращения: 02.05.2026).
23. Spring Data JPA Reference Documentation [Электронный ресурс] / Broadcom Inc. — 2026. — URL: <https://docs.spring.io/spring-data/jpa/reference/index.html> (дата обращения: 02.05.2026).
24. Hibernate ORM User Guide [Электронный ресурс]. — Режим доступа: https://docs.hibernate.org/stable/orm/userguide/html_single/ (дата обращения: 02.05.2026).
25. iText Core: Developer Documentation [Электронный ресурс] / iText Group NV. — 2026. — URL: <https://itextpdf.com/resources/api-documentation> (дата обращения: 02.05.2026).
26. PostgreSQL 18 Documentation [Электронный ресурс] / PostgreSQL Global Development Group. — 2026. — URL: <https://www.postgresql.org/docs/18/index.html> (дата обращения: 02.05.2026).
27. Apache Kafka Documentation [Электронный ресурс] / Apache Software Foundation. — 2026. — URL: <https://kafka.apache.org/documentation> (дата обращения: 02.05.2026).
28. Spring Framework Reference. WebSockets :: Spring Framework [Электронный ресурс] / Broadcom Inc. — 2026. — URL: <https://docs.spring.io/spring/reference/web/websocket.html> (дата обращения: 02.05.2026).
29. Docker Documentation [Электронный ресурс] / Docker Inc. — 2026. — URL: <https://docs.docker.com> (дата обращения: 02.05.2026).
30. ПАО «ГМК «Норильский никель». Схема производства металлов [Электронный ресурс]. — Режим доступа: <https://ar2022.nornickel.ru/ru/business-overview/production-flow.html> (дата обращения: 25.04.2026).

services:

data-mart-frontend:

build:

context: ./Frontend

dockerfile: Dockerfile

image: data-mart-frontend

container_name: data-mart-frontend

ports:

- 5173:5173

depends_on:

- data-mart-backend

networks:

- network

data-generator:

build:

context: ./DataGenerator

dockerfile: Dockerfile

image: data-generator

container_name: data-generator

environment:

GENERATOR_PORT: \${GENERATOR_PORT}

SPRING_KAFKA_BOOTSTRAP_SERVERS:

\${SPRING_KAFKA_BOOTSTRAP_SERVERS}

KAFKA_MES_TOPIC: \${KAFKA_MES_TOPIC}

KAFKA_MES_GROUP: \${KAFKA_MES_GROUP}

KAFKA_SCADA_TOPIC: \${KAFKA_SCADA_TOPIC}

KAFKA_SCADA_GROUP: \${KAFKA_SCADA_GROUP}

KAFKA_LIMS_TOPIC: \${KAFKA_LIMS_TOPIC}

KAFKA_LIMS_GROUP: \${KAFKA_LIMS_GROUP}

ports:

- 8080:8080

depends_on:

- kafka

networks:

- network

data-mart-backend:

build:

context: ./DataMartBackend

dockerfile: Dockerfile

image: data-mart-backend

container_name: data-mart-backend

environment:

BACKEND_PORT: \${BACKEND_PORT}

SPRING_KAFKA_BOOTSTRAP_SERVERS:

\${SPRING_KAFKA_BOOTSTRAP_SERVERS}

KAFKA_MES_TOPIC: \${KAFKA_MES_TOPIC}

KAFKA_MES_GROUP: \${KAFKA_MES_GROUP}

KAFKA_SCADA_TOPIC: \${KAFKA_SCADA_TOPIC}

KAFKA_SCADA_GROUP: \${KAFKA_SCADA_GROUP}

KAFKA_LIMS_TOPIC: \${KAFKA_LIMS_TOPIC}

KAFKA_LIMS_GROUP: \${KAFKA_LIMS_GROUP}

DB_URL: \${DB_URL}

DB_NAME: \${DB_NAME}

SPRING_DATASOURCE_USERNAME: \${DB_USER}

SPRING_DATASOURCE_PASSWORD: \${DB_PASSWORD}

SPRING_JPA_HIBERNATE_DDL_AUTO: update

JWT_SECRET: \${JWT_SECRET}

ports:

- 8081:8081

depends_on:

- postgres

- kafka

networks:

- network

postgres:

build:

context: ./Postgres

dockerfile: Dockerfile

image: data-mart-db

environment:

POSTGRES_DB: \${DB_NAME}

POSTGRES_USER: \${DB_USER}

POSTGRES_PASSWORD: \${DB_PASSWORD}

volumes:

- postgres-data:/var/lib/postgresql/18/docker

- ./Postgres/init.sql:/docker-entrypoint-initdb.d/init.sql

healthcheck:

test: ["CMD-SHELL", "pg_isready -U \${POSTGRES_USER} -d
\${POSTGRES_DB}"]

interval: 5s

timeout: 5s

retries: 5

expose:

- 5432

networks:

- network

kafka:

image: confluentinc/cp-kafka:7.4.4

environment:

KAFKA_BROKER_ID: 1

KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181

KAFKA_ADVERTISED_LISTENERS:

PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:29092

KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:

PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT

KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT

KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

expose:

- 9092

healthcheck:

test: kafka-topics.sh --list --bootstrap-server

\${SPRING_KAFKA_BOOTSTRAP_SERVERS} || exit 1

interval: 1s

timeout: 10s

retries: 10

depends_on:

- zookeeper

networks:

- network

zookeeper:

image: confluentinc/cp-zookeeper:7.4.4

environment:

ZOOKEEPER_CLIENT_PORT: 2181

ZOOKEEPER_TICK_TIME: 2000

expose:

- 2181

networks:

- network

networks:

network:

driver: bridge

volumes:

postgres-data:

CREATE EXTENSION IF NOT EXISTS pg_cron;

CREATE TABLE IF NOT EXISTS fact_notifications (

id SERIAL PRIMARY KEY,

message TEXT,

equipment_id TEXT,

sensor_id TEXT,

signal_source TEXT,

severity TEXT,

status TEXT,

comment TEXT,

viewed BOOLEAN DEFAULT false,

```
created_at TIMESTAMP DEFAULT now(),  
updated_at TIMESTAMP,  
updated_by TEXT  
);
```

```
CREATE TABLE IF NOT EXISTS dim_batch (  
    batch_id TEXT PRIMARY KEY,  
    metal_type TEXT NOT NULL,  
    start_time TIMESTAMP,  
    processing_time TIMESTAMP,  
    analyses_time TIMESTAMP,  
    end_time TIMESTAMP,  
    process_status TEXT,  
    output_yield DOUBLE PRECISION  
);
```

```
CREATE TABLE IF NOT EXISTS fact_mes (  
    record_id TEXT PRIMARY KEY,  
    order_id TEXT NOT NULL,  
    batch_id TEXT NOT NULL REFERENCES dim_batch(batch_id),  
    equipment_id TEXT NOT NULL,  
    operator_id TEXT,  
    charge_mass DOUBLE PRECISION,  
    output_mass DOUBLE PRECISION,  
    duration_min INT  
);
```

```
CREATE TABLE IF NOT EXISTS fact_lims (  
    record_id TEXT PRIMARY KEY,
```

```
batch_id TEXT NOT NULL REFERENCES dim_batch(batch_id),
sample_id TEXT NOT NULL,
analysis_method TEXT,
test_date TIMESTAMP,
status TEXT
);
```

```
CREATE TABLE IF NOT EXISTS fact_lims_results (
  id SERIAL PRIMARY KEY,
  record_id TEXT NOT NULL REFERENCES fact_lims(record_id),
  parameter_name TEXT,
  value TEXT,
  unit TEXT,
  normal BOOLEAN DEFAULT true
);
```

```
CREATE TABLE IF NOT EXISTS fact_scada (
  record_id TEXT NOT NULL,
  sensor_id TEXT NOT NULL,
  equipment_id TEXT NOT NULL,
  time TIMESTAMP NOT NULL,
  parameter TEXT,
  value DOUBLE PRECISION,
  unit TEXT,
  status TEXT,
  PRIMARY KEY (record_id, time)
) PARTITION BY RANGE (time);
```

```
CREATE TABLE IF NOT EXISTS users (
```

```
user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
username TEXT UNIQUE NOT NULL,
password TEXT NOT NULL,
name TEXT,
surname TEXT,
patronymic TEXT,
email TEXT,
role TEXT NOT NULL,
created_at TIMESTAMP DEFAULT NOW()
);
```

```
CREATE INDEX IF NOT EXISTS idx_notifications_viewed
ON fact_notifications(viewed);
```

```
CREATE INDEX IF NOT EXISTS idx_notifications_created_at
ON fact_notifications(created_at DESC);
```

```
CREATE INDEX IF NOT EXISTS idx_notifications_viewed_created
ON fact_notifications(viewed, created_at DESC);
```

```
CREATE INDEX IF NOT EXISTS idx_batch_time_range
ON dim_batch(start_time, end_time);
```

```
CREATE INDEX IF NOT EXISTS idx_batch_status
ON dim_batch(process_status);
```

```
CREATE INDEX IF NOT EXISTS idx_mes_batch
ON fact_mes(batch_id);
```

```
CREATE INDEX IF NOT EXISTS idx_mes_equipment  
ON fact_mes(equipment_id);
```

```
CREATE INDEX idx_mes_batch_equipment_covering  
ON fact_mes (batch_id)  
INCLUDE (equipment_id);
```

```
CREATE INDEX IF NOT EXISTS idx_mes_batch_equipment  
ON fact_mes(batch_id, equipment_id);
```

```
CREATE INDEX IF NOT EXISTS idx_lims_batch  
ON fact_lims(batch_id);
```

```
CREATE INDEX IF NOT EXISTS idx_lims_test_date  
ON fact_lims(test_date DESC);
```

```
CREATE INDEX IF NOT EXISTS idx_lims_batch_test_date  
ON fact_lims(batch_id, test_date DESC);
```

```
CREATE INDEX IF NOT EXISTS idx_lims_results_record  
ON fact_lims_results(record_id);
```

```
CREATE INDEX IF NOT EXISTS idx_scada_equipment  
ON fact_scada(equipment_id);
```

```
CREATE INDEX IF NOT EXISTS idx_scada_equipment_time_covering  
ON fact_scada (equipment_id, time)  
INCLUDE (parameter, value, unit, status);
```

```
CREATE INDEX IF NOT EXISTS idx_scada_time_brin
ON fact_scada USING brin (time);
```

```
CREATE INDEX IF NOT EXISTS idx_scada_anomaly_status
ON fact_scada (equipment_id, time)
WHERE status IN ('WARNING', 'ALARM');
```

```
CREATE INDEX IF NOT EXISTS idx_dim_batch_dashboard_stats
ON dim_batch (process_status, metal_type);
```

```
INSERT INTO users
```

```
(username, password, name, surname, patronymic, email, role)
```

```
VALUES
```

```
('admin',
```

```
'$2a$10$p3rjHEHq5vlfZ9mJhPt2mudzuqT.AaDi0nTyLITIrXmKZreBn9izq',
```

```
'Анна', 'Парамонова', 'Сергеевна', 'admin@gmail.com', 'ADMIN'),
```

```
('manager',
```

```
'$2a$10$p3rjHEHq5vlfZ9mJhPt2mudzuqT.AaDi0nTyLITIrXmKZreBn9izq',
```

```
'Никита', 'Жерновников', 'Олегович', 'manager@gmail.com', 'MANAGEMENT'),
```

```
('lab',
```

```
'$2a$10$p3rjHEHq5vlfZ9mJhPt2mudzuqT.AaDi0nTyLITIrXmKZreBn9izq',
```

```
'Ольга', 'Волкова', 'Станиславовна', 'lab@gmail.com', 'LABORATORY'),
```

```
('prod',
```

```
'$2a$10$p3rjHEHq5vlfZ9mJhPt2mudzuqT.AaDi0nTyLITIrXmKZreBn9izq',
```

```
'Сергей', 'Рыбаков', 'Александрович', 'prod@gmail.com', 'PRODUCTION');
```

```
CREATE TABLE fact_scada_default
```

```
PARTITION OF fact_scada DEFAULT;
```

```

CREATE TABLE fact_scada_2026_05
    PARTITION OF fact_scada
    FOR VALUES FROM ('2026-05-01') TO ('2026-06-01');

CREATE OR REPLACE FUNCTION create_next_scada_partition()
RETURNS void AS $$
DECLARE
    next_month DATE := DATE_TRUNC('month', NOW() + INTERVAL '1
month');
    partition_name TEXT;
    start_date TEXT;
    end_date TEXT;
BEGIN
    partition_name := 'fact_scada_' || TO_CHAR(next_month, 'YYYY_MM');
    start_date := TO_CHAR(next_month, 'YYYY-MM-DD');
    end_date := TO_CHAR(next_month + INTERVAL '1 month', 'YYYY-
MM-DD');

    IF NOT EXISTS (
        SELECT 1 FROM pg_class WHERE relname = partition_name
    ) THEN
        EXECUTE FORMAT(
            'CREATE TABLE %I PARTITION OF fact_scada FOR VALUES
FROM (%L) TO (%L)',
            partition_name, start_date, end_date
        );
    END IF;
END;
$$ LANGUAGE plpgsql;

```

```
SELECT cron.schedule('create-scada-partition', '0 9 25 * *',
  'SELECT create_next_scada_partition());
import axios from 'axios';
```

```
const API = axios.create({
  baseURL: 'http://localhost:8081/api'
});
```

```
API.interceptors.request.use(config => {
  const token = localStorage.getItem('token');
  if (token) {
    config.headers.Authorization =
      `Bearer ${token}`;
  }
  return config;
});
```

```
API.interceptors.response.use(
  (response) => {
    const newToken = response.headers['x-new-token'];
    if (newToken) {
      localStorage.setItem('token', newToken);
    }
    return response;
  },
  (error) => {
    if (error.response?.status === 401) {
      localStorage.removeItem('token');
    }
  }
);
```

```

        window.location.replace('/login');
    }
    return Promise.reject(error);
}
);

```

// АВТОРИЗАЦИЯ

```

export const loginUser = (data) => API.post('/auth/login', data);
export const registerUser = (data) => API.post('/auth/register', data);

```

// УВЕДОМЛЕНИЯ

```

export const getActiveNotifications = () => API.get('/notifications/active');
export const markNotificationInProgress = (id) =>
API.post(`/notifications/${id}/in-progress`);
export const getAllNotifications = (dto) => API.post('/notifications', dto);
export const updateNotification = (id, dto) => API.put(`/notifications/${id}`,
dto);
export const getNotificationsStats = () => API.get('/notifications/stats');

```

// АДМИН

```

export const getUsers = (filter) => API.post('/admin/users', filter);
export const updateUser = (id, data) => API.put(`/admin/users/${id}`, data);

```

// ОБЩИЕ

```

export const getMetalCards = () => API.get('/metal-cards');
export const getMetalBatches = (filter) => API.post('/metals', filter);
export const getBatch = (batchId) => API.get(`/batches/${batchId}`);
export const getMesByBatch = (batchId) => API.get(`/mes/${batchId}`);
export const getLimsWithoutResultsByBatch = (batchId) =>

```

```

API.get(`/lims/${batchId}`);

export const getCurrentUser = () => API.get('/users/me');

export const getScadaAvgByBatch = (batchId) =>
API.get(`/scada/${batchId}`);

export const createReport = (batchId) => API.get(`/report/${batchId}`, {
  responseType: 'blob' });

```

// ЛАБОРАТОРИЯ

```

export const getLastLims = () => API.get('/laboratory/last-lims');

export const getLimsByBatch = (batchId) =>
API.get(`/laboratory/lims/${batchId}`);

```

// ПРОИЗВОДСТВО

```

export const getLastBatches = () => API.get('/production/last-batches');

export const getScadaByBatch = (batchId) =>
API.get(`/production/scada/${batchId}`);

```

// МЕНЕДЖМЕНТ

```

export const getMetalStatisticsCards = () => API.get('/management/metal-
statistics-cards');

```